Interpretability

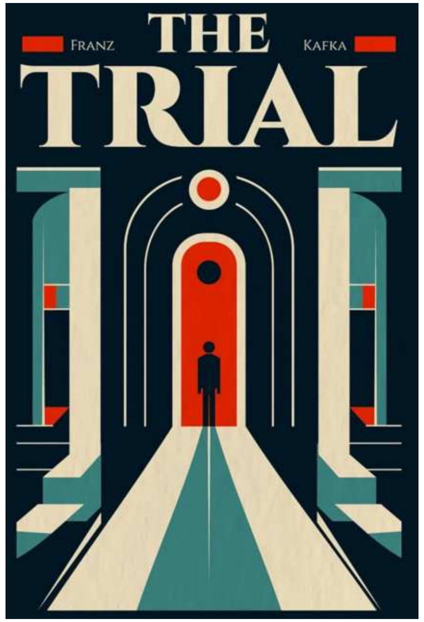
ACTL3143 & ACTL5111 Deep Learning for Actuaries

Patrick Laub

Lecture Outline

The right to an explanation

> On your thirtieth birthday, you awake to find two police officers standing over you. They inform you that you're under arrest. When you ask what for, they tell you that they don't know, but their boss, the inspector, will be along soon, and he'll be able to tell you. A few hours later, you're ushered into a neighboring tenant's bedroom. The inspector tells you that he doesn't know what you've been charged with, but you're free to go about your life, except that you must report to a hearing on Sunday. And so you move through the criminal justice system, never knowing what crime you've allegedly committed, nor understanding the rules of the process. On the eve of your thirty-first birthday, you're taken away and executed.
>
> Fortunately, this story is not about you, but is instead about Josef K., the fictional protagonist of Kafka's The Trial.
>
> — Vredenburgh (2022, p. 1)



> Unfortunately, a broadly similar story is probably true about you, thanks to the use of algorithms to aid decision-making in a variety of institutions. One such example comes from Washington, DC. In 2009, the city set out to improve its school system, where only 8 percent of eighth graders were performing at grade level in math. It introduced IMPACT, a model to evaluate teacher performance developed by Mathematica Policy Research. The aim of the model was to identify poorly performing teachers, as input to firing decisions. The model that Mathematica Policy Research developed was complex and proprietary, and teachers couldn't find anyone to explain to them why they'd been fired. In response to the firing of her colleagues on the basis of the model's outputs, Sarah Bax, a math teacher, asked: "How do you justify evaluating people by a measure for which you are unable to provide an explanation?".
>
> — Vredenburgh (2022, p. 1)

Kate Vredenburgh

The situation in insurance

> "Understanding the relationships between the model inputs and outputs builds additional confidence in any model but is particularly challenging for many types of machine learning models. And in many applications, it is necessary for users and reviewers of models to understand whether the relationships between the model inputs and outputs make business sense, are compliant with laws and regulations, and/or can be explained to individuals impacted by the use of the model."
>
> — Baeder et al. (2021, p. 5)

UNSW SYDNEY

ⓘ Article 22 GDPR – Automated individual decision-making, including profiling

1. The data subject shall have the right **not** to be subject to a decision based solely on automated processing, including profiling, which produces legal effects concerning him or her or similarly significantly affects him or her.
2. Paragraph 1 shall not apply if the decision:
    a. is necessary for entering into, or performance of, a contract between the data subject and a data controller;
    b. is authorised by Union or Member State law to which the controller is subject and which also lays down suitable measures to safeguard the data subject's rights and freedoms and legitimate interests; or
    c. is based on the data subject's explicit consent.
3. In the cases referred to in points (a) and (c) of paragraph 2, the data controller shall implement suitable measures to safeguard the data subject's rights and freedoms and legitimate interests, at least the right to obtain human intervention on the part of the controller, to express his or her point of view and to contest the decision.
4. Decisions referred to in paragraph 2 shall not be based on special categories of personal data referred to in Article 9(1), unless point (a) or (g) of Article 9(2) applies and suitable measures to safeguard the data subject's rights and freedoms and legitimate interests are in place.

Definitions

> "Surprisingly enough, although the concept of interpretability is increasingly widespread, there is no general consensus on both the definition and the measurement of the interpretability of a model."
>
> — Delcaillau et al. (2022)

> "**Definition** Interpret means *to explain or to present in understandable terms*. In the context of ML systems, we define interpretability as *the ability to explain or to present in understandable terms to a human*."
>
> — Doshi-Velez & Kim (2017)

UNSW SYDNEY

Interpretability ≠ Explainability

> "Interpretability is about transparency, about understanding exactly why and how the model is generating predictions, and therefore, it is important to observe the inner mechanics of the algorithm considered. This leads to interpreting the model's parameters and features used to determine the given output. Explainability is about explaining the behavior of the model in human terms."
>
> — Charpentier (2024)

UNSW SYDNEY

Aspects of interpretability

Inherent Interpretability

The model is interpretable by design.

Post-hoc Explanations

The model is not interpretable by design, but we can use other methods to explain the model.

Global Interpretability

The ability to understand how the model works.

Local Interpretability

The ability to interpret/understand each prediction.

Interpretable insurance pricing

For general insurance pricing, we adopted six interpretability and practical requirements:

1. *Transparency of main effects* — an exact description of each covariate's univariate effect on the output.
2. *Transparency of interaction effects* — which interactions are used, and how they influence the prediction.
3. *Quantification of variable importance* — rank the covariates by their influence on the output.
4. *Sparsity* — only include features with a significant impact on the prediction.
5. *Monotonicity* — capture known monotone relationships (e.g. worse bonus–malus $\Rightarrow$ higher premium).
6. *Smoothness* — predictions should vary smoothly with certain rating factors.

Source: Laub et al. (2026)

UNSW SYDNEY

Lecture Outline

Package imports

```python
import random

import matplotlib.pyplot as plt
import numpy as np
import numpy.random as rnd
import pandas as pd

import geopandas as gpd
import contextily as ctx
from shapely.geometry import Point
import plotly.graph_objects as go

import keras
from keras.metrics import SparseTopKCategoricalAccuracy
from keras.models import Sequential, Model
from keras.layers import Dense, Input
from keras.callbacks import EarlyStopping

from sklearn import tree, set_config
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, StandardScaler, OneHotEnc
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.compose import make_column_transformer
from sklearn.metrics import mean_poisson_deviance, mean_absolute_error
```

Husky vs. Wolf

(a) Husky classified as wolf (b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the "Husky vs Wolf" task.

A well-known anecdote in the explainability literature (Ribeiro et al., 2016).

First attempt at NLP task

```python
df_raw["SUMMARY_EN"]
```

```
0       V1, a 2000 Pontiac Montana minivan,
made a lef...
1       The crash occurred in the eastbound
lane of a  ...
2       This crash occurred just after the
noon time h...
                              ...
6946    The crash occurred in the eastbound
lanes of a...
6947    This single-vehicle crash occurred in
a rural  ...
6948    This two vehicle daytime collision
occurred mi...
Name: SUMMARY_EN, Length: 6949, dtype: str
```

```python
df_raw["NUM_VEHICLES"].value_counts()\
  .sort_index()
```

```
NUM_VEHICLES
1      1822
2      4151
3+      976
Name: count, dtype: int64
```

Bag of words for the top 1,000 words

```python
vect = CountVectorizer(max_features=1_000, stop_words="english")
vect.fit(X_train["SUMMARY_EN"])

X_train_bow = vectorise_dataset(X_train, vect)
X_val_bow = vectorise_dataset(X_val, vect)
X_test_bow = vectorise_dataset(X_test, vect)

vectorise_dataset(X_train, vect, dataframe=True).head()
```

	10	105	113	12	15	150	16	17	18	180	...	yield	zone	WEATHER1
2532	0	0	0	0	0	0	0	0	0	0	...	0	0	0
6209	0	0	0	0	0	0	0	0	0	0	...	0	0	0
2561	1	0	1	0	0	0	0	0	0	0	...	0	0	0
6664	0	0	0	0	0	0	0	0	0	0	...	0	0	0
4214	0	0	0	0	0	0	0	0	0	0	...	0	0	0

5 rows × 1008 columns

Trained a basic neural network on that

```python
num_features = X_train_bow.shape[1]
num_cats = df_raw["NUM_VEHICLES"].nunique()
model = build_model(num_features, num_cats)
es = EarlyStopping(patience=1, restore_best_weights=True, monitor="val_accuracy")
model.fit(X_train_bow, y_train, epochs=10,
    callbacks=[es], validation_data=(X_val_bow, y_val), verbose=0)
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 100)	100,900
dense_1 (Dense)	(None, 3)	303

Total params: 303,611 (1.16 MB)
Trainable params: 101,203 (395.32 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 202,408 (790.66 KB)

```python
model.evaluate(X_train_bow, y_train, verbose=0)
```

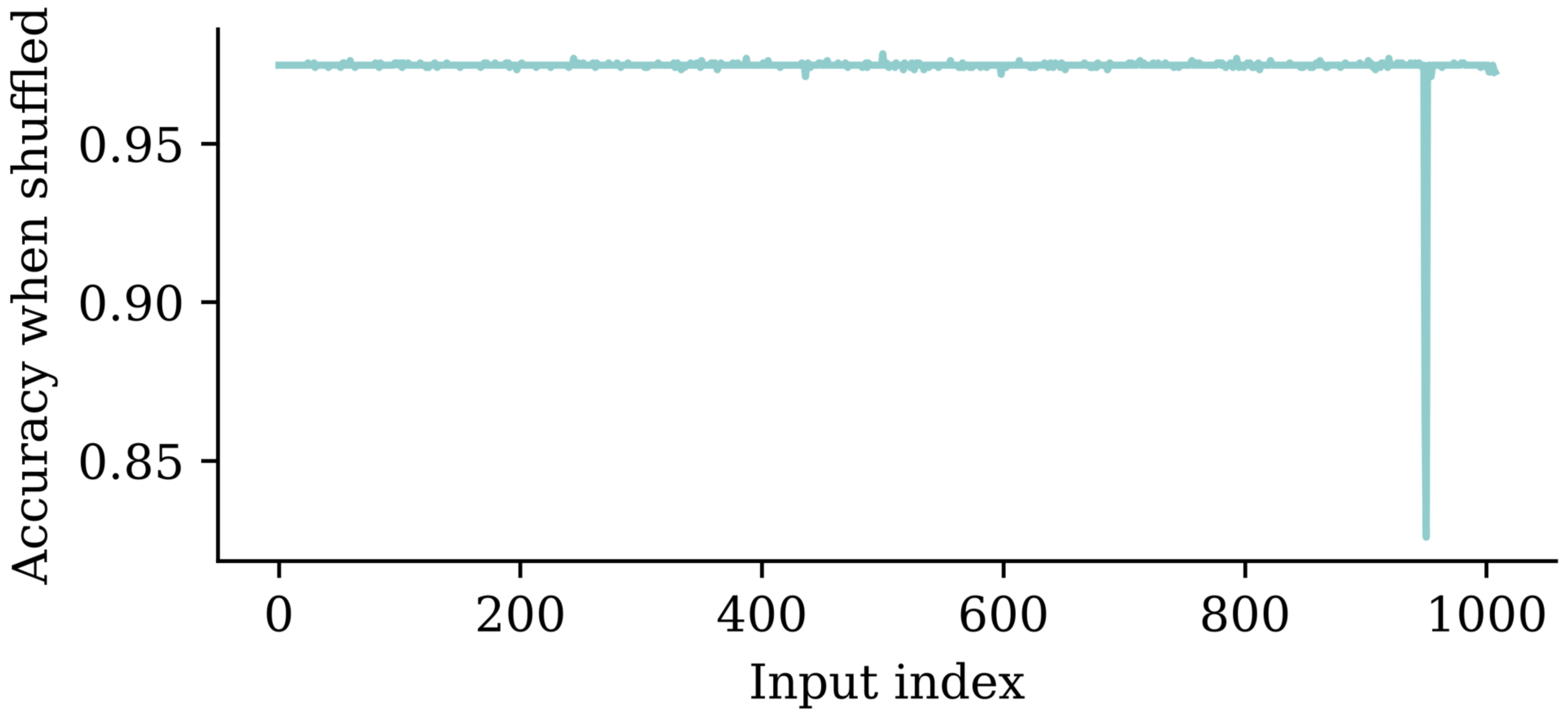
```
[0.03331480547785759, 0.9949628114700317, 0.9997601509094238]
```

```python
model.evaluate(X_val_bow, y_val, verbose=0)
```

```
[0.09635408222675323, 0.9748201370239258, 0.9978417158126831]
```

Run the permutation test

```
all_perm_scores = permutation_test(model, X_val_bow, y_val)
```



Find the most significant inputs

```
['v3', 'v2', 'vehicle', 'harmful', 'v4', 'motor', 'WEATHER8', 'WEATHER5', 'WEATHER4',
'WEATHER3', 'posted', 'trailer', 'parked', 'drove', 'chevrolet', 'event', 'single',
'WEATHER7', 'lane', 'light', 'legally', 'steered', 'traffic', 'continued', 'rest', 'concrete',
'afternoon', 'surface', 'arterial', 'asphalt', 'coded', 'second', 'seizure', 'seven',
'shortly', 'change', 'sightline', 'sign', 'struck', 'signs', 'striking', 'braking', 'began',
'spun', 'started', 'steady', 'attack', 'researcher', 'directions', 'decision', 'medication',
'median', 'maneuver', 'male', 'make', 'gas', 'meters', 'located', 'heading', 'leaving',
'kilometers', 'involved', 'inadequate', 'information', 'limit', 'crossed', 'minivan', 'mph',
'pre', 'pole', 'plane', 'direction', 'pickup', 'drivers', 'movement', 'pain', 'ordered',
'oncoming', 'encroaching', 'observed', 'number', 'noticed', 'dry', 'travel', 'injuries',
'2006', 'waiting', 'year', 'vehicles', 'utility', '30', 'treatment', 'WEATHER1', '38', '48',
'ability', 'notice', 'noted', 'numerous', 'object']
```

How about a simple decision tree?

```python
clf = tree.DecisionTreeClassifier(random_state=0, max_leaf_nodes=3)
clf.fit(X_train_bow[:, best_input_inds], y_train);
```

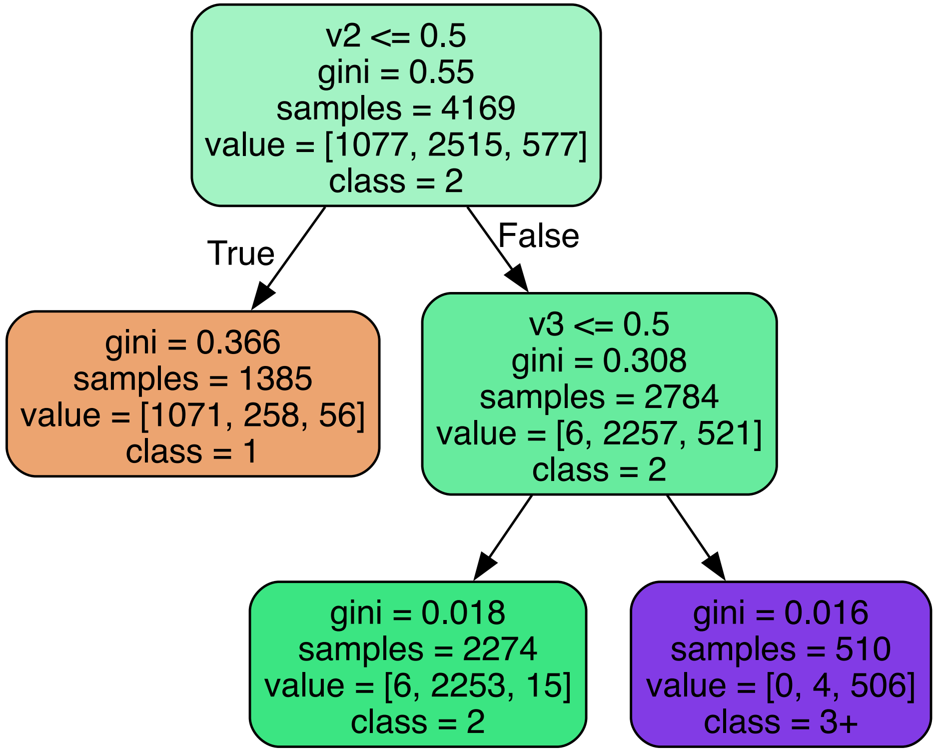
```python
print(clf.score(X_train_bow[:, best_input_inds], y_train))
print(clf.score(X_val_bow[:, best_input_inds], y_val))
```

```
0.9186855360997841
0.9316546762589928
```

The decision tree ends up giving pretty good results.

Decision tree

```python
tree.plot_tree(clf, feature_names=best_inputs, filled=True);
```




```python
print(np.where(clf.feature_importances_ > 0)[0])
[best_inputs[ind] for ind in np.where(clf.feature_importances_ > 0)[0]]
```

```
[0 1]
['v3', 'v2']
```

This is why we replace "v1", "v2", "v3"

There was a slide in the NLP deck titled "Just ignore this for now…" That was going through each summary and replacing the words "V1", "V2", "V3" with random numbers. This was done to see if the model was overfitting to these words.

```
model.fit(X_train_bow, y_train, epochs=10,
    callbacks=[es], validation_data=(X_val_bow, y_val), verbose=0);
```

Retraining on the fixed dataset gives us a more realistic (lower) accuracy.

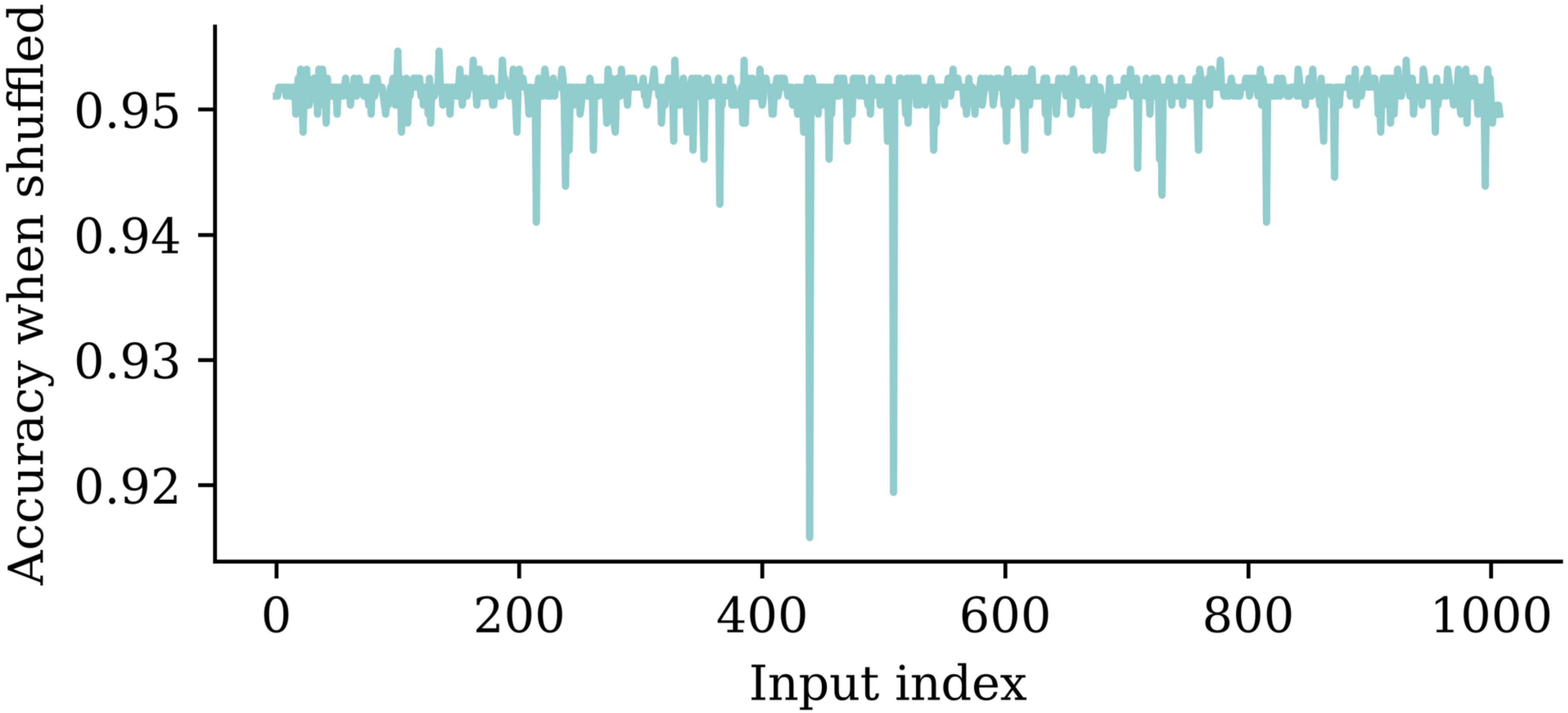
```
model.evaluate(X_train_bow, y_train, verbose=0)
```

```
[0.022723009809851646, 0.9990405440330505, 1.0]
```

```
model.evaluate(X_val_bow, y_val, verbose=0)
```

```
[0.16461113095283508, 0.9517985582351685, 0.9964028596878052]
```

Permutation importance accuracy plot

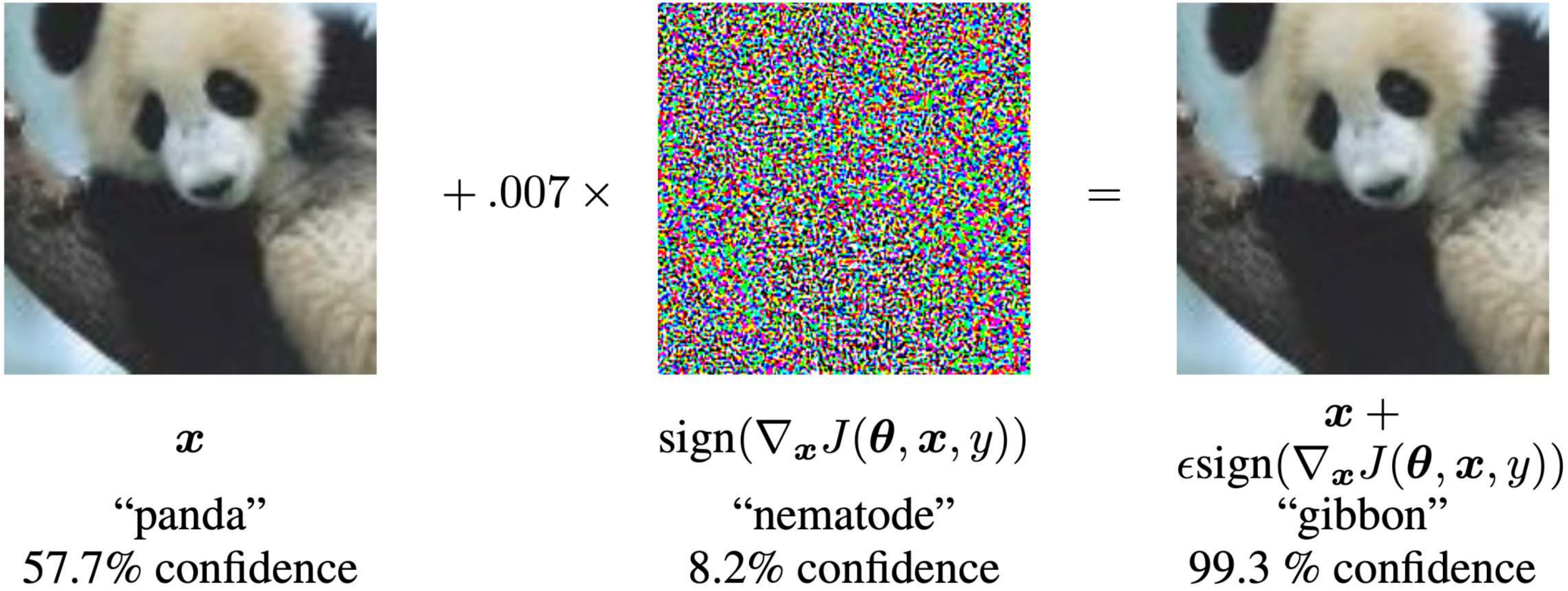


Find the most significant inputs

```python
vocab = vect.get_feature_names_out()
input_cols = list(vocab) + weather_cols

best_input_inds = np.argsort(perm_scores)[:100]
best_inputs = [input_cols[idx] for idx in best_input_inds]

print(best_inputs)
```

```
['harmful', 'involved', 'single', 'coded', 'event', 'reason', 'year', 'contacted', 'struck',
'pushed', 'hit', 'encroachment', 'rear', 'continued', 'road', 'pickup', 'non', 'pole',
'limit', 'edge', 'critical', 'driven', 'impact', 'motor', 'intersection', 'stopped', 'police',
'guardrail', 'dry', 'vehicles', 'chevrolet', 'daylight', 'afternoon', '1999', 'occurred',
'traffic', 'day', 'alcohol', 'failure', 'WEATHER2', 'associated', 'westbound', 'familiar',
'interview', '30', 'lane', 'line', 'dodge', 'traveling', 'daily', 'away', 'encroaching',
'corner', 'asphalt', 'clear', 'ford', 'kph', 'grand', 'WEATHER5', 'WEATHER4', 'work',
'weekday', 'undivided', 'treated', 'toyota', 'stop', 'ran', 'poor', 'forward', 'parked',
'occupants', 'nissan', 'median', 'male', 'interviewed', 'injured', 'inadequate', 'impairment',
'home', 'heard', 'opposite', 'according', 'WEATHER8', '37', '1993', '23', '72', 'drives',
'right', 'contacting', '2003', 'contributed', 'unsuccessful', 'performance', 'paved',
'behavior', 'roof', 'brake', '46', 'initial']
```

Adversarial examples



A demonstration of fast adversarial example generation applied to GoogLeNet on ImageNet. By adding an imperceptibly small vector whose elements are equal to the sign of the elements of the gradient of the cost function with respect to the input, we can change GoogLeNet's classification of the image (Goodfellow et al., 2014)

Lecture Outline

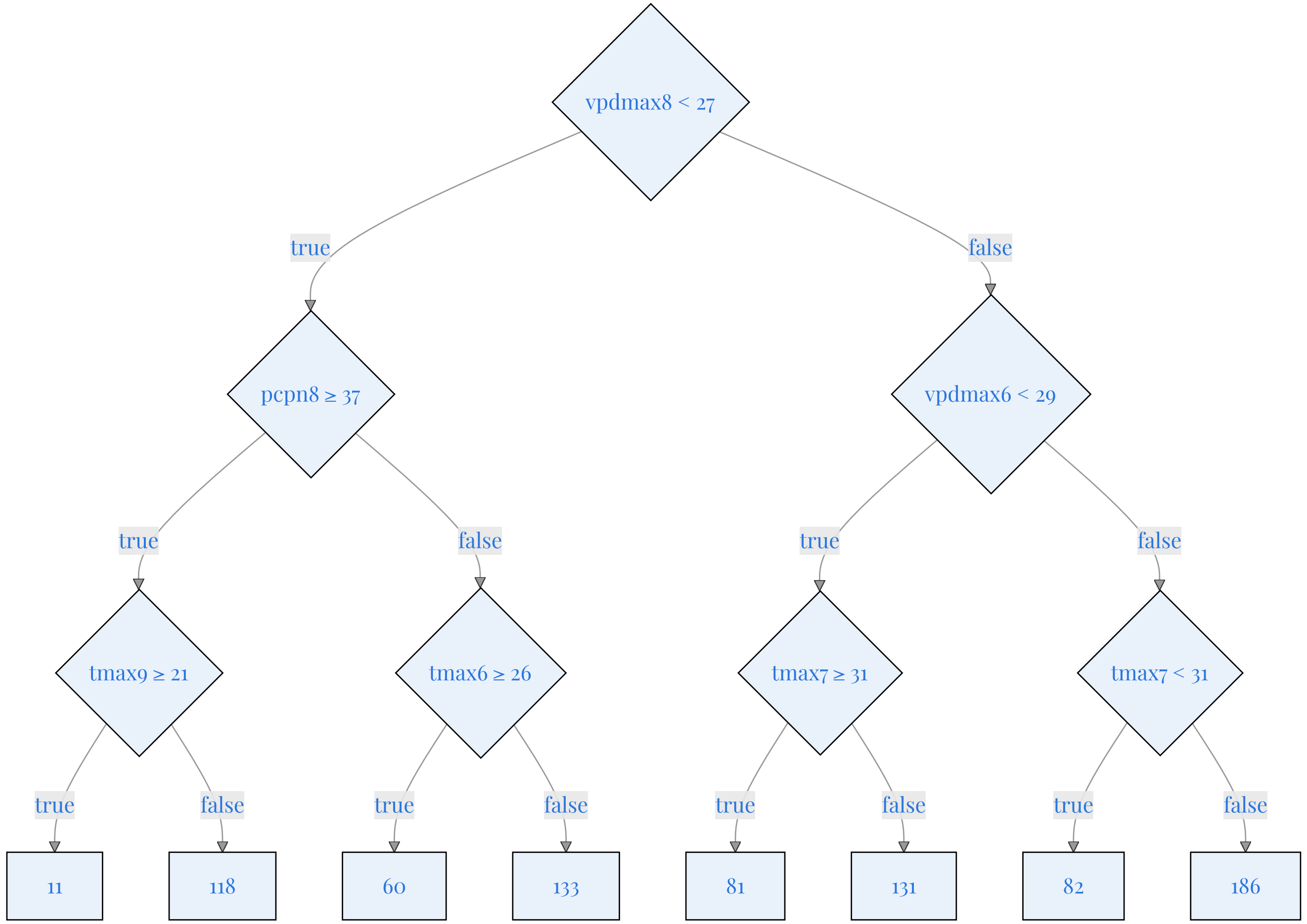
What is inherent interpretability?

> "Interpretability by design is decided on the level of the machine learning algorithm. If you want a machine learning algorithm that produces interpretable models, the algorithm has to constrain the search of models to those that are interpretable. The simplest example is linear regression: When you use ordinary least squares to fit/train a linear regression model, you are using an algorithm that will produce … models that are linear in the input features. Models that are interpretable by design are also called **intrinsically** or **inherently** interpretable models."
>
> — Molnar (2020)

Christoph Molnar

Examples of interpretable models

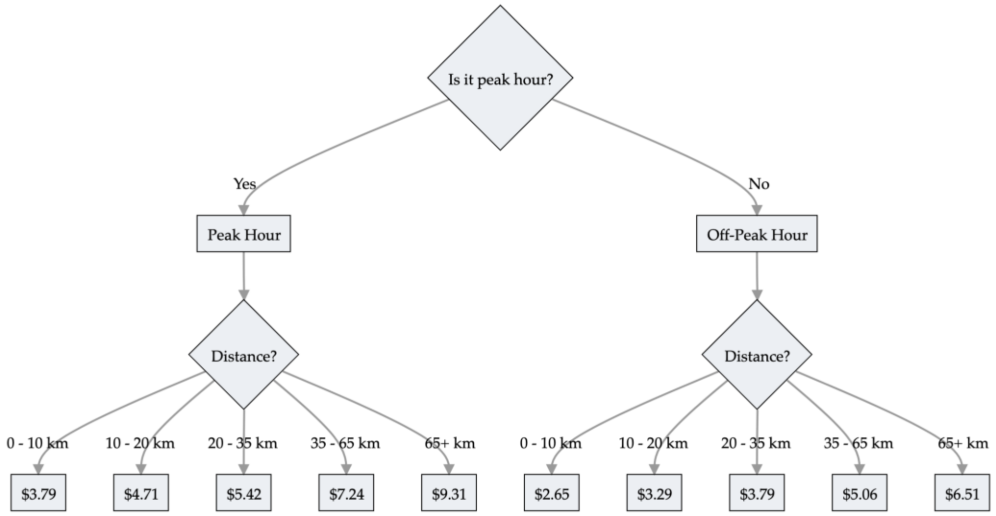
- Linear regression
- Generalised linear models
- Decision trees
- Decision rules



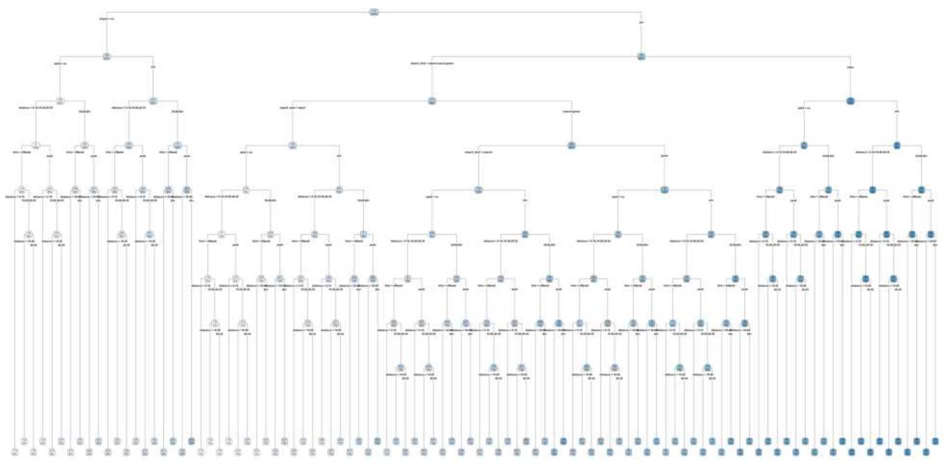
E.g. decision tree for payouts of index insurance (Chen et al., 2024).

See, e.g., Carreira-Perpinán & Tavallali (2018), for modern trees.

Some processes don't fit the tree structure



Train prices



Full train pricing

Decision rules

Make predictions using if-then statements related to the inputs.

> "Decision rules can be **as expressive as decision trees, while being more compact**. Decision trees often also suffer from replicated sub-trees, that is, when the splits in a left and a right child node have the same structure."
>
> — Molnar (2020)

```python
def rail_cost(peak_hours, distance):
  if peak_hours:
      if distance ⩽ 10:
        cost = 3.79
      elif distance ⩽ 20:
        cost = 4.71
      elif distance ⩽ 35:
        cost = 5.42
      elif distance ⩽ 65:
        cost = 7.24
      else:
        cost = 9.31
  else:
      if distance ⩽ 10:
        cost = 2.65
      elif distance ⩽ 20:
        cost = 3.29
      elif distance ⩽ 35:
        cost = 3.79
      elif distance ⩽ 65:
        cost = 5.06
      else:
        cost = 6.51
  return cost
```

UNSW SYDNEY

Generalised linear & additive models

A generalised linear model has the form

$$\hat{y} = g^{-1}\big(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p\big)$$

where $\beta_0, \ldots, \beta_p$ are the model parameters.

Global & local interpretations are easy to obtain.

A *Generalised Additive Model* replaces each linear term with a flexible *shape function* f_j:

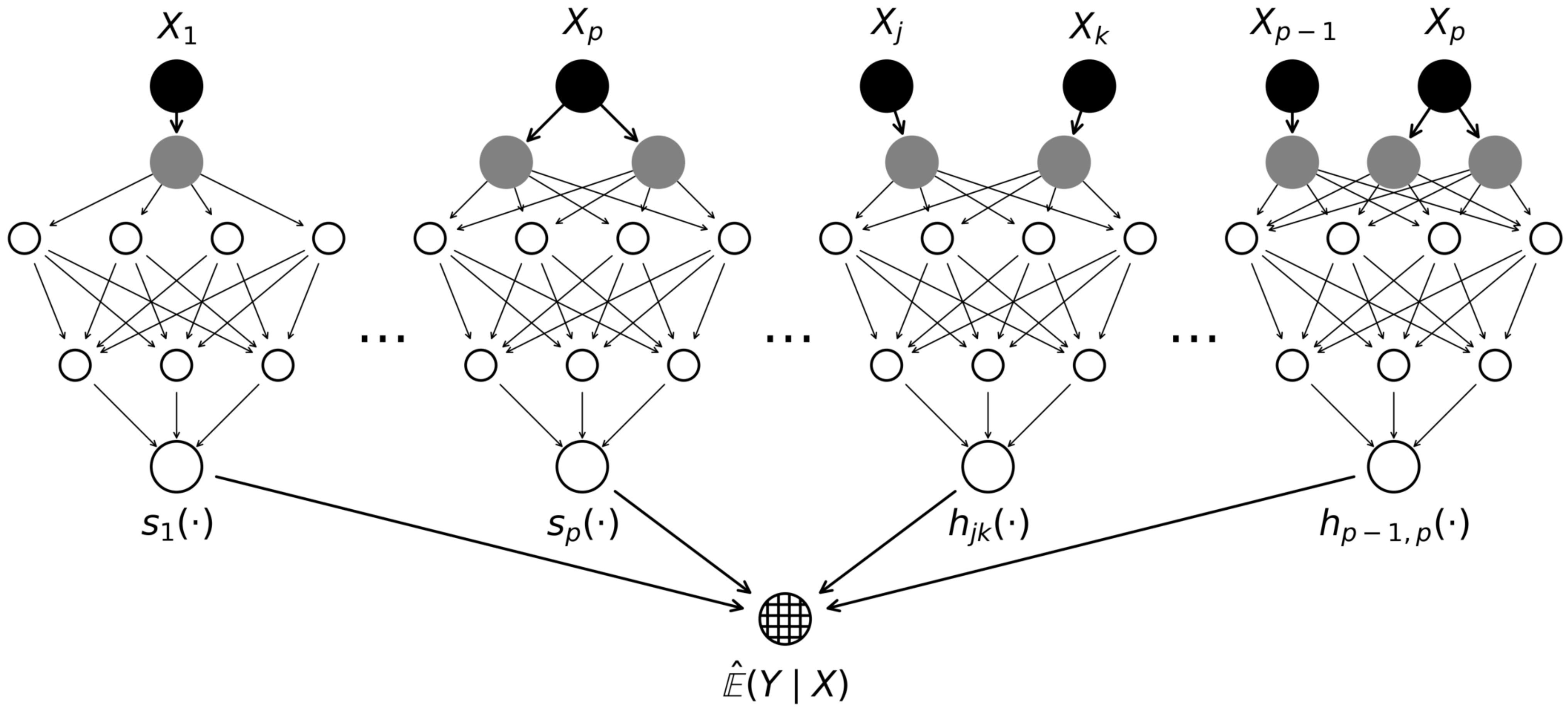
$$\hat{y} = g^{-1}\big(\beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_p(x_p)\big)$$

The f_j are typically splines. Each covariate's effect is still a curve you can plot, so the model stays transparent while capturing non-linear effects.

UNSW SYDNEY

Neural Additive Models (NAMs)

A NAM is a GAM where each shape function f_j is a *small neural network*, learned jointly with the rest of the model (Agarwal et al., 2021).



Each covariate (or select interactions) receives its own subnetwork, which contributes additively.

Source: Laub et al. (2026, fig. 1).

Actuarial NAMs (ANAMs)

Recall the six requirements for an interpretable pricing model — the ANAM is a NAM engineered around that checklist (Laub et al., 2026).

Requirement	**Mechanism in ANAM**
Transparency of main effects	one subnetwork per covariate, plotted as a *shape function*
Transparency of interactions	dedicated subnetworks for the selected pairs only
Variable importance	variance of each shape function over the data
Sparsity	explicit selection of features and interactions
Monotonicity	enforced by the architecture
Smoothness	roughness penalty on the shape functions

LocalGLMNet

A different route to interpretability: keep the GLM's *form*, but let the coefficients vary (Richman & Wüthrich, 2023).

$$\hat{y}_i = g^{-1}\big(\beta_0(\boldsymbol{x}_i) + \beta_1(\boldsymbol{x}_i)x_{i1} + \cdots + \beta_p(\boldsymbol{x}_i)x_{ip}\big)$$

A GLM with local parameters $\beta_0(\boldsymbol{x}_i), \ldots, \beta_p(\boldsymbol{x}_i)$ for each observation $\boldsymbol{x}_i$. The local parameters are the output of a neural network.

Lecture Outline

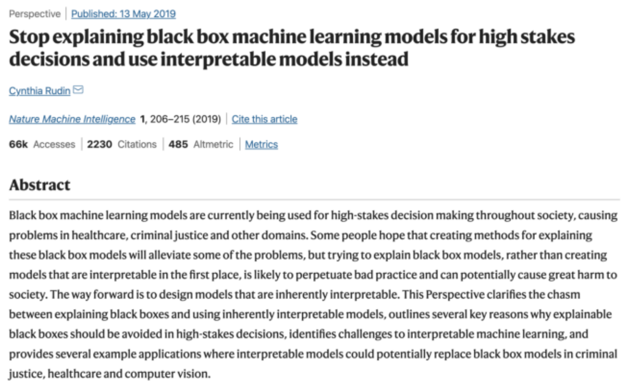
- Introduction
- Models With False Accuracy
- Inherently Interpretable Models
- **Post-hoc Explanations**
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Other Local Methods
- Conclusion

Explaining a black box

> A black box machine learning model is a formula that is either too complicated for any human to understand, or proprietary, so that one cannot understand its inner workings. Black box models are difficult to troubleshoot, which is particularly problematic for medical data. Black box models often predict the right answer for the wrong reason (the "Clever Hans" phenomenon), leading to excellent performance in training but poor performance in practice. There are numerous other issues with black box models. In criminal justice, individuals may have been subjected to years of extra prison time due to typographical errors in black box model inputs and poorly-designed proprietary models for air quality have had serious consequences for public safety during wildfires...
>
> — Rudin et al. (2022, pp. 2–3)

Cynthia Rudin

> Determining whether a black box model is fair with respect to gender or racial groups is much more difficult than determining whether an interpretable model has such a bias… Explaining black boxes, rather than replacing them with interpretable models, can make the problem worse by providing misleading or false characterizations, or adding unnecessary authority to the black box. There is a clear need for innovative machine learning models that are inherently interpretable.
>
> — Rudin et al. (2022, pp. 2–3)


Perspective | Published: 13 May 2019

Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead

Cynthia Rudin

Nature Machine Intelligence **1**, 206–215 (2019) | Cite this article

66k Accesses | **2230** Citations | **485** Altmetric | Metrics

Abstract

Black box machine learning models are currently being used for high-stakes decision making throughout society, causing problems in healthcare, criminal justice and other domains. Some people hope that creating methods for explaining these black box models will alleviate some of the problems, but trying to explain black box models, rather than creating models that are interpretable in the first place, is likely to perpetuate bad practice and can potentially cause great harm to society. The way forward is to design models that are inherently interpretable. This Perspective clarifies the chasm between explaining black boxes and using inherently interpretable models, outlines several key reasons why explainable black boxes should be avoided in high-stakes decisions, identifies challenges to interpretable machine learning, and provides several example applications where interpretable models could potentially replace black box models in criminal justice, healthcare and computer vision.

Rudin (2019)

Permutation importance

- Inputs: fitted model m, tabular dataset D.
- Compute the reference score s of the model m on data D (for instance the accuracy for a classifier or the R^2 for a regressor).
- For each feature j (column of D):
  - For each repetition k in $1, \ldots, K$:
    - Randomly shuffle column j of dataset D to generate a corrupted version of the data named $\tilde{D}_{k,j}$.
    - Compute the score $s_{k,j}$ of model m on corrupted data $\tilde{D}_{k,j}$.
  - Compute importance i_j for feature f_j defined as:

$$i_j = s - \frac{1}{K}\sum_{k=1}^{K} s_{k,j}$$

Originally proposed by Breiman (2001), extended by Fisher et al. (2019).

Source: scikit-learn documentation, permutation_importance function.

UNSW SYDNEY

Permutation importance

```python
def permutation_test(model, X, y, num_reps=1, seed=42, batch_size=8192):
    """
    Run the permutation test for variable importance.
    Returns matrix of shape (X.shape[1], len(model.evaluate(X, y))).
    """
    rnd.seed(seed)
    scores = []

    for j in range(X.shape[1]):
        original_column = np.copy(X[:, j])
        col_scores = []

        for r in range(num_reps):
            rnd.shuffle(X[:,j])
            col_scores.append(model.evaluate(X, y, verbose=0, batch_size=batch_size))

        scores.append(np.mean(col_scores, axis=0))
        X[:,j] = original_column

    return np.array(scores)
```

Three different plots

A *Ceteris Paribus* plot shows how the model's prediction changes as we vary the value of *one covariate/input* while keeping *all other features constant* at their original values for some observation.

An Individual Conditional Expectation (ICE) plot is the same thing as a ceteris paribus plot, but for *all observations* in the dataset.

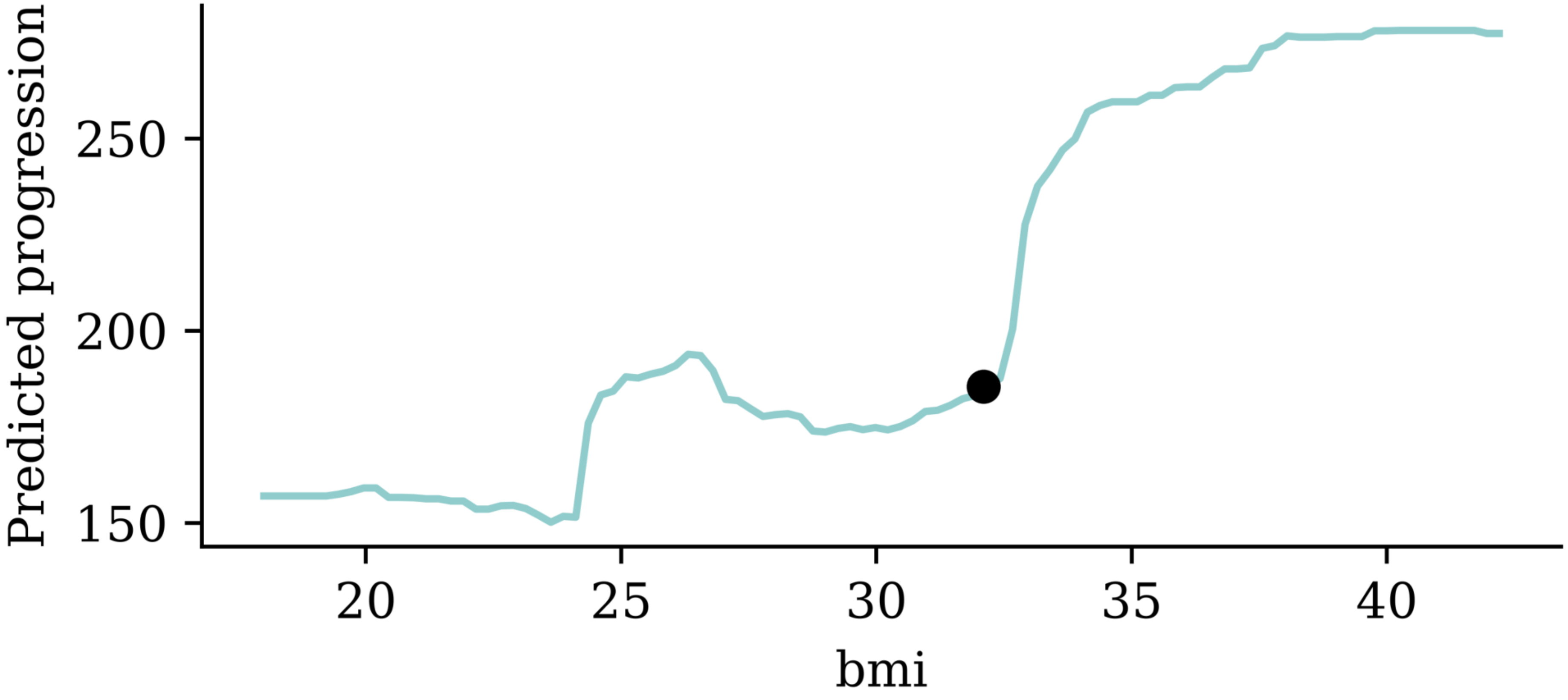
A Partial Dependence Plot (PDP) shows how the model's prediction changes as we vary the value of *one covariate/input* while averaging over all other features in the dataset.

All of these show the relationship between an input and the prediction that the *model* has learned — not necessarily the true relationship in the world.

See Molnar (2020, sec. 12). Note, there are documented issues with PDPs; they can even be manipulated to hide discrimination (Xin et al., 2025).

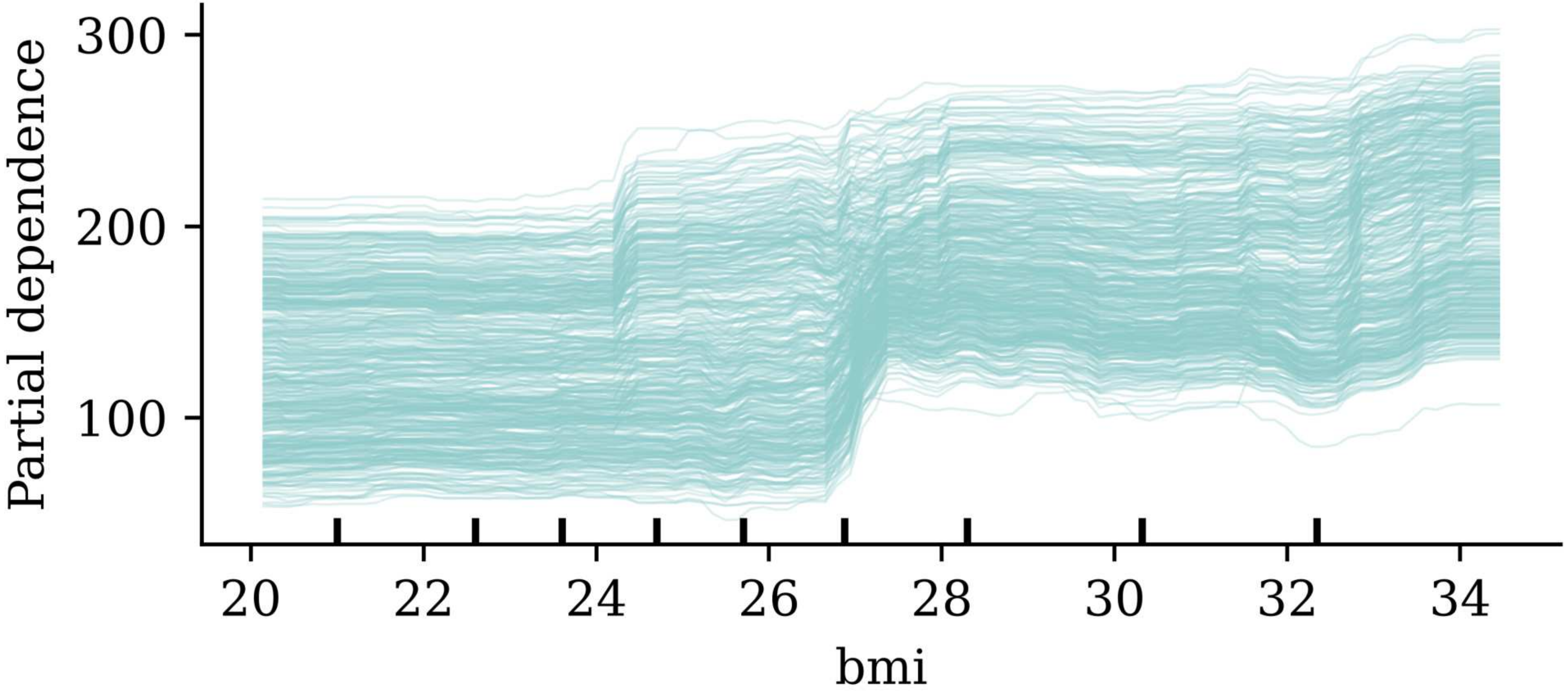
Example: ceteris paribus plot

Fit a random forest to predict diabetes progression, take *one patient*, and vary their `bmi` while holding *every other feature fixed*.



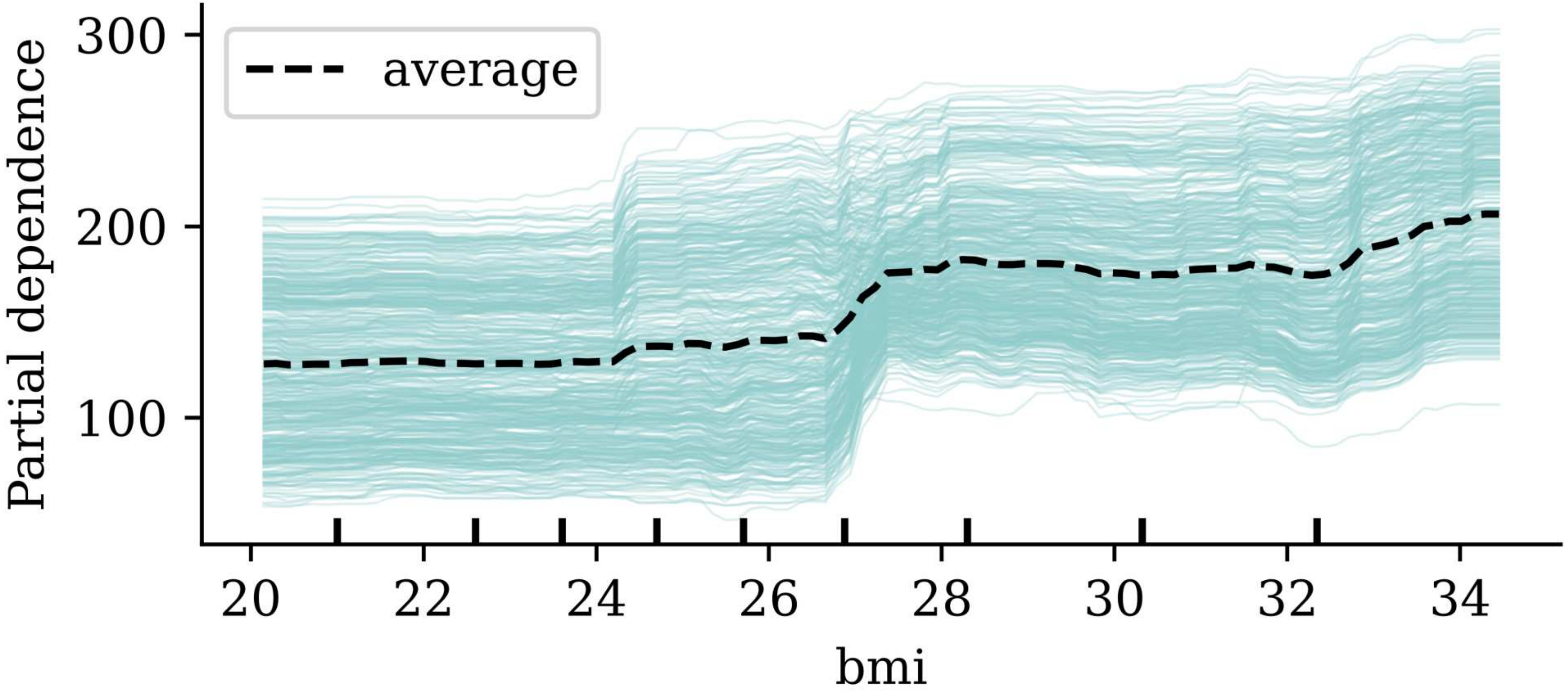
Example: ICE plot

Draw the same ceteris paribus curve for *every* observation in the dataset.



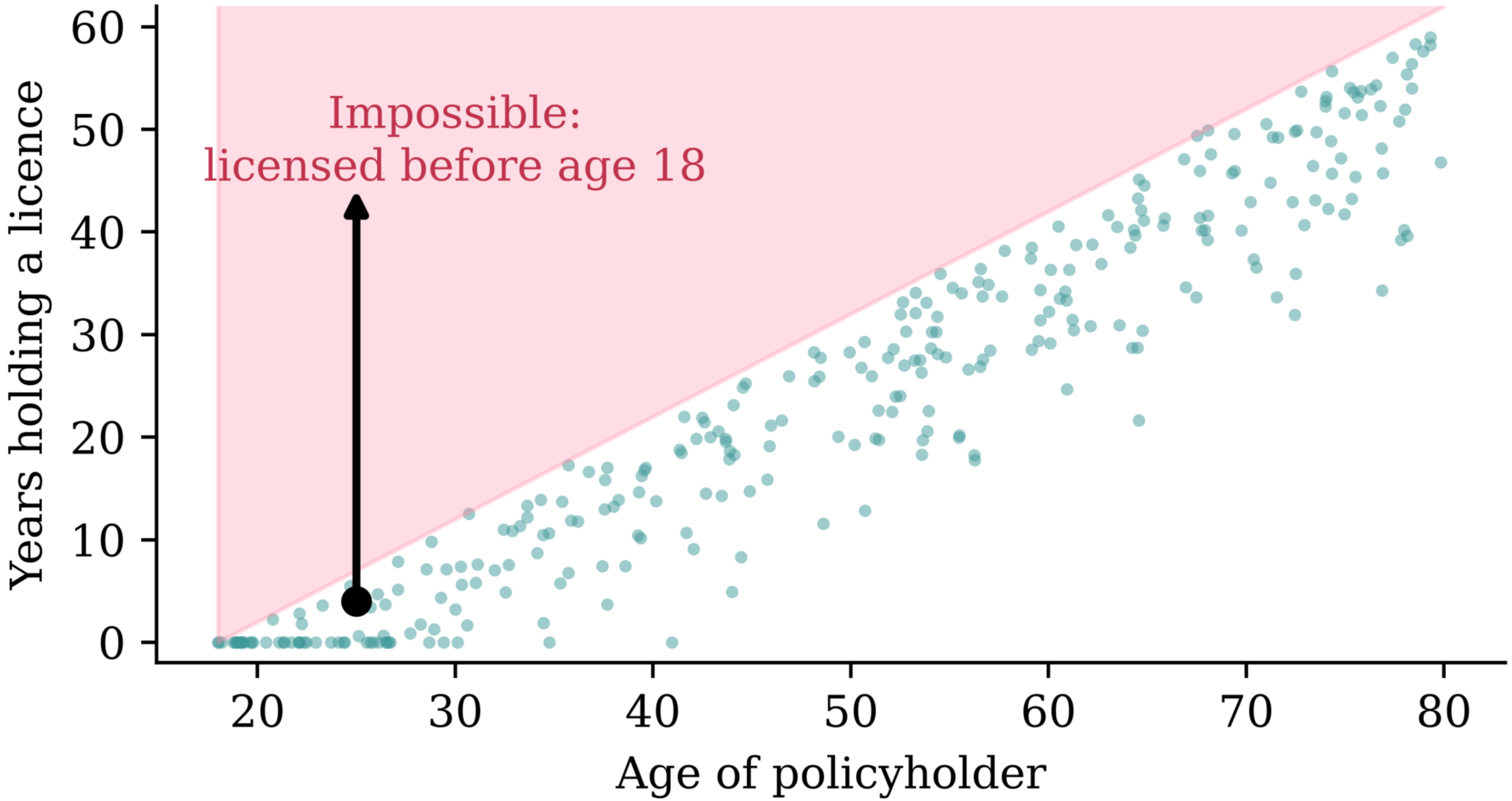
Example: partial dependence plot

Averaging the ICE curves gives the partial dependence plot.



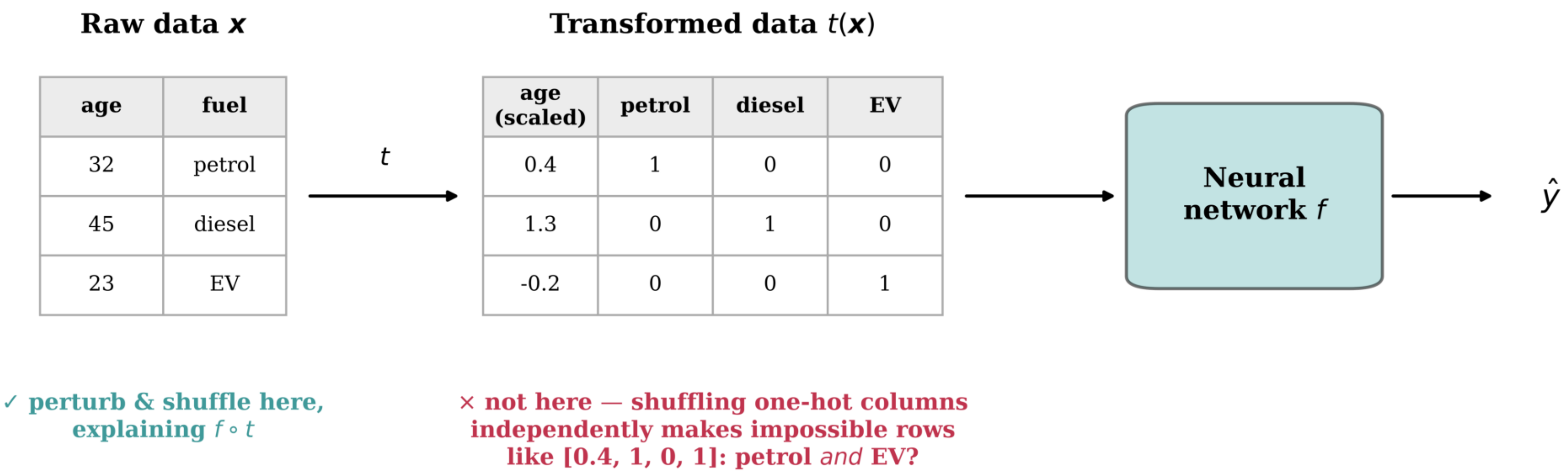
Warning: correlated features

These methods (and permutation importance) change *one feature at a time*, leaving the others untouched — so they can manufacture inputs that could never exist.



Explain the model on the raw features

The model to explain is the *composition* $f \circ t$; perturb the *raw* features $\boldsymbol{x}$ and re-apply the preprocessing t, rather than perturbing $t(\boldsymbol{x})$.



Lecture Outline

beMTPL97 dataset

```
data = pd.read_csv('data/raw/beMTPL97.csv')
claims = data.drop(columns = ["id", "claim", "amount", "average"])
claims.shape
```

```
(163212, 14)
```

```
postcode = claims.pop("postcode")
train_raw, test_raw = train_test_split(claims, test_size=0.2, random_state=2000, stratify

X_train_raw = train_raw.drop(columns='nclaims')
X_test_raw = test_raw.drop(columns='nclaims')
y_train_raw = train_raw['nclaims']
y_test_raw = test_raw['nclaims']

int_cols = X_train_raw.select_dtypes(include='integer').columns
X_train_raw[int_cols] = X_train_raw[int_cols].astype(float)
X_test_raw[int_cols] = X_test_raw[int_cols].astype(float)

num_vars = ['expo', 'ageph', 'bm', 'power', 'agec', 'lat', 'long']
cat_vars = ['coverage', 'sex', 'fuel', 'use', 'fleet']
```

Covariates

Numerical variables:

Variable	Description
`expo`	exposure to risk.
`ageph`	policyholder's age.
`bm`	An integer for the level on the former Belgian bonus-malus scale (0 to 22; higher = worse claims history).
`power`	vehicle's horsepower in kilowatts.
`agec`	vehicle's age in years.
`long`	longitude of the policyholder's municipality center.
`lat`	latitude of the policyholder's municipality center.

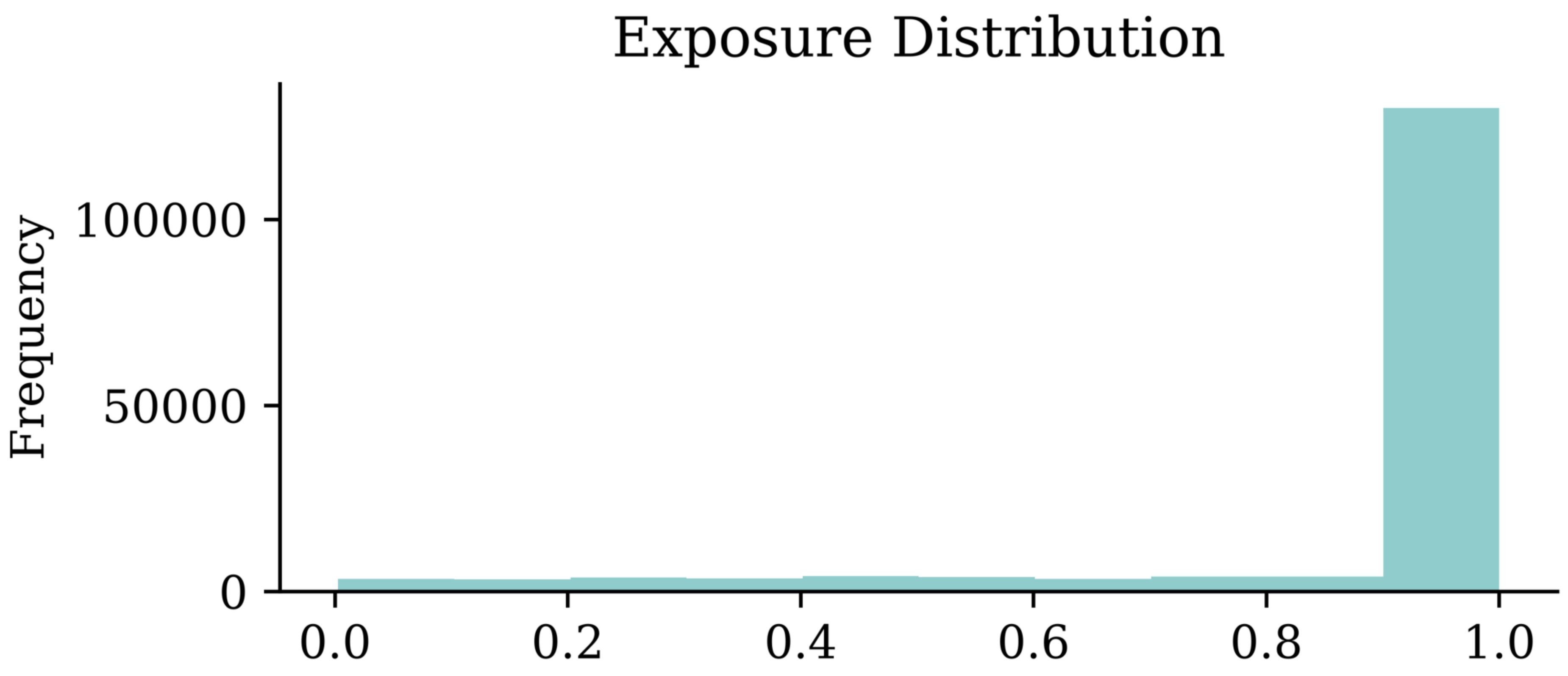
Target is `nclaims`.

Categorical variables:

Variable	Description
`coverage`	insurance coverage level: `"TPL"` (third party liability), `"TPL+"` (TPL + limited material damage), `"TPL++"` (TPL + comprehensive material damage).
`sex`	policyholder's gender: `"female"`, `"male"`.
`fuel`	vehicle's fuel type: `"gasoline"` or `"diesel"`.
`use`	vehicle's use: `"private"` or `"work"`.
`fleet`	vehicle is part of a fleet (`1` = yes, `0` = no).

Exposure & frequency

```
claims["expo"].plot(kind='hist', title='Exposure Distribution')
```




```
claims["nclaims"].value_counts()
```

```
nclaims
0    144936
1     16539
2      1556
```

In-class exercise

Consider:

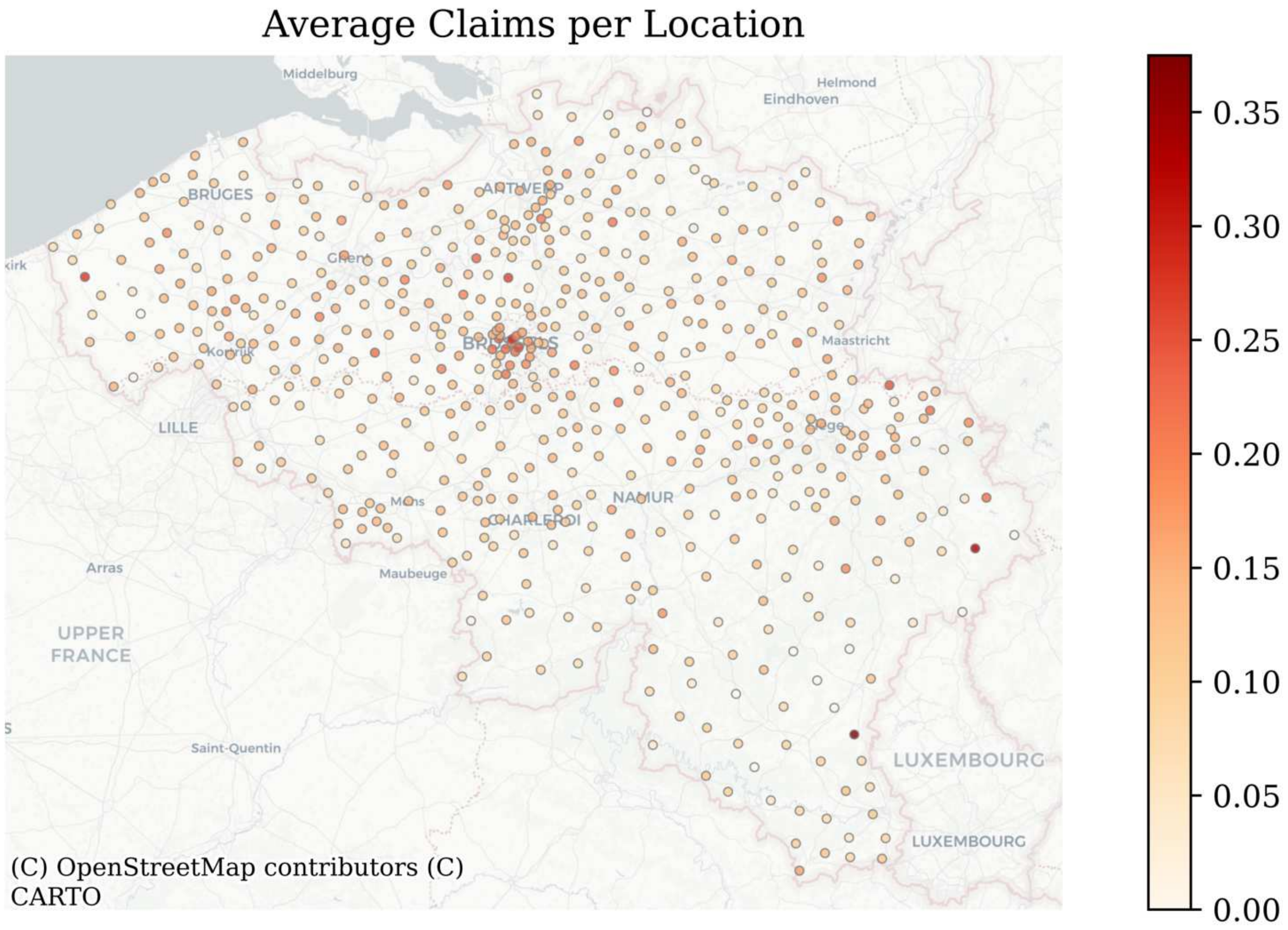
- exposure
- policyholder age
- bonus–malus level
- vehicle age or power
- geographical location

For at least three variables:

1. Describe or sketch the relationship you would expect with the predicted number of claims.
2. Decide whether the relationship should be imposed by the model, learned freely from the data, or merely checked after fitting.
3. Identify one result that would make you distrust the model, even if its predictive performance were good.

UNSW SYDNEY

Map of claims



Fitting using statsmodels

```python
formula = "nclaims ~ expo + ageph + bm + power + agec + lat + long " + \
    " + C(coverage) + C(sex) + C(fuel) + C(use) + C(fleet)"

glm_model = smf.glm(
    formula=formula,
    data=train_raw,
    family=sm.families.Poisson()
).fit()
```

Question

What do you expect to be the relationship between `ageph` and `nclaims`?

```python
X_train_first = X_train_raw.iloc[0:1].copy()
X_train_first
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet
25776	1.0	TPL	59.0	female	0.0	51.0	15.0	gasoline	private	0.0

Summary

```python
glm_model.summary().tables[0]
```

Generalized Linear Model Regression Results

Dep. Variable:	nclaims	**No. Observations:**	130569
Model:	GLM	**Df Residuals:**	130555
Model Family:	Poisson	**Df Model:**	13
Link Function:	Log	**Scale:**	1.0000
Method:	IRLS	**Log-Likelihood:**	-49908.
Date:	Sun, 12 Jul 2026	**Deviance:**	69690.
Time:	22:04:54	**Pearson chi2:**	1.39e+05
No. Iterations:	6	**Pseudo R-squ. (CS):**	0.01842
Covariance Type:	nonrobust		

Summary

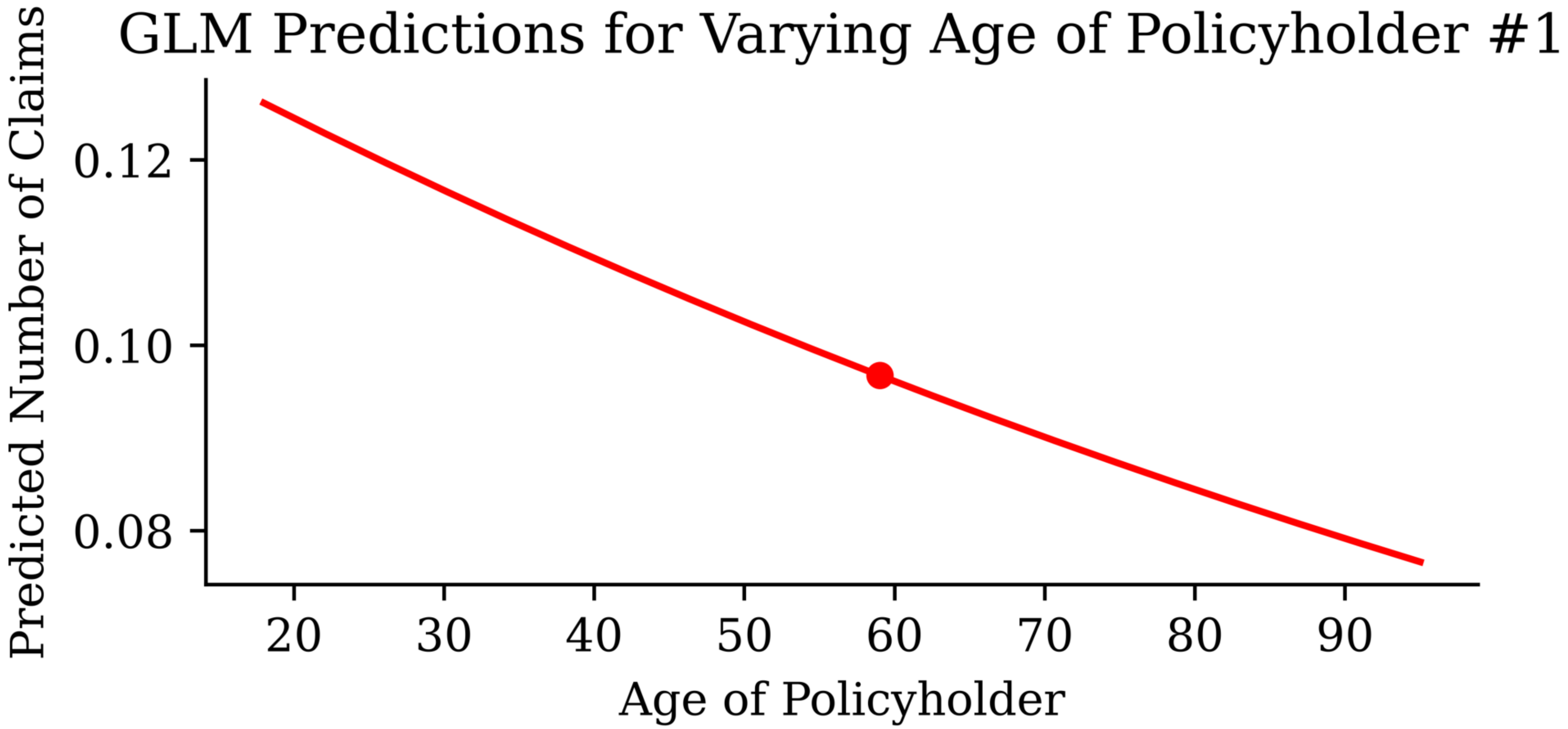
```
glm_model.summary().tables[1]
```

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	-7.0204	1.344	-5.222	0.000	-9.655	-4.386
C(coverage)[T.TPL+]	-0.0765	0.020	-3.889	0.000	-0.115	-0.038
C(coverage)[T.TPL++]	-0.0715	0.027	-2.660	0.008	-0.124	-0.019
C(sex)[T.male]	-0.0259	0.018	-1.429	0.153	-0.061	0.010
C(fuel)[T.gasoline]	-0.1793	0.017	-10.459	0.000	-0.213	-0.146
C(use)[T.work]	-0.0864	0.037	-2.306	0.021	-0.160	-0.013
C(fleet)[T.1]	-0.0886	0.048	-1.832	0.067	-0.183	0.006
expo	0.9893	0.041	24.196	0.000	0.909	1.069
ageph	-0.0065	0.001	-10.724	0.000	-0.008	-0.005
bm	0.0642	0.002	33.035	0.000	0.060	0.068
power	0.0036	0.000	8.309	0.000	0.003	0.004
agec	-0.0019	0.002	-0.872	0.383	-0.006	0.002
lat	0.0777	0.026	2.969	0.003	0.026	0.129
long	0.0279	0.011	2.542	0.011	0.006	0.049

Lecture Outline

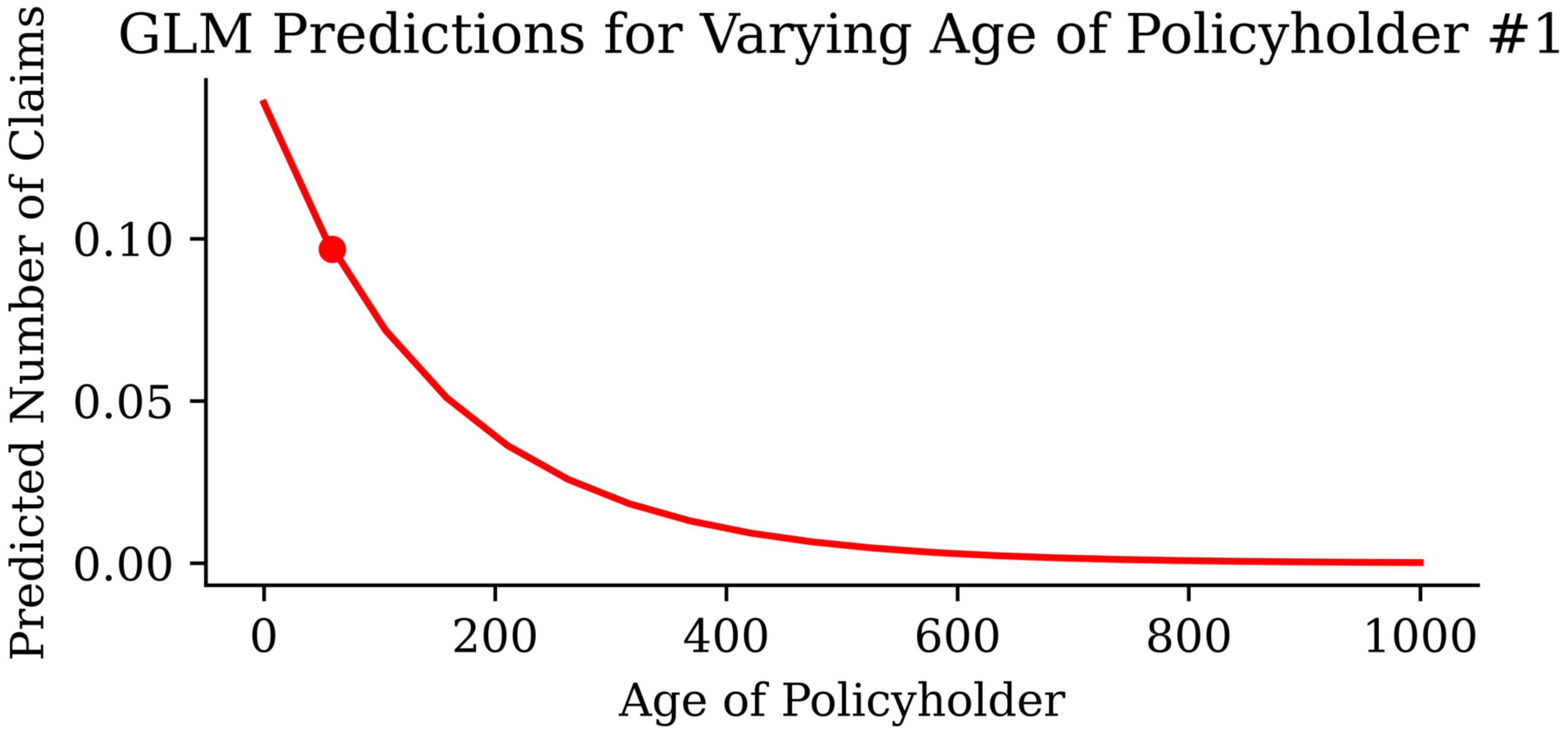
GLM Ceteris Paribus Plots

```python
ageph_range = np.linspace(train_raw['ageph'].min(), train_raw['ageph'].max(), 20)
y_pred_batch = []
for ageph in ageph_range:
    X_train_first['ageph'] = ageph
    y_pred_batch.append(glm_model.predict(X_train_first))
```



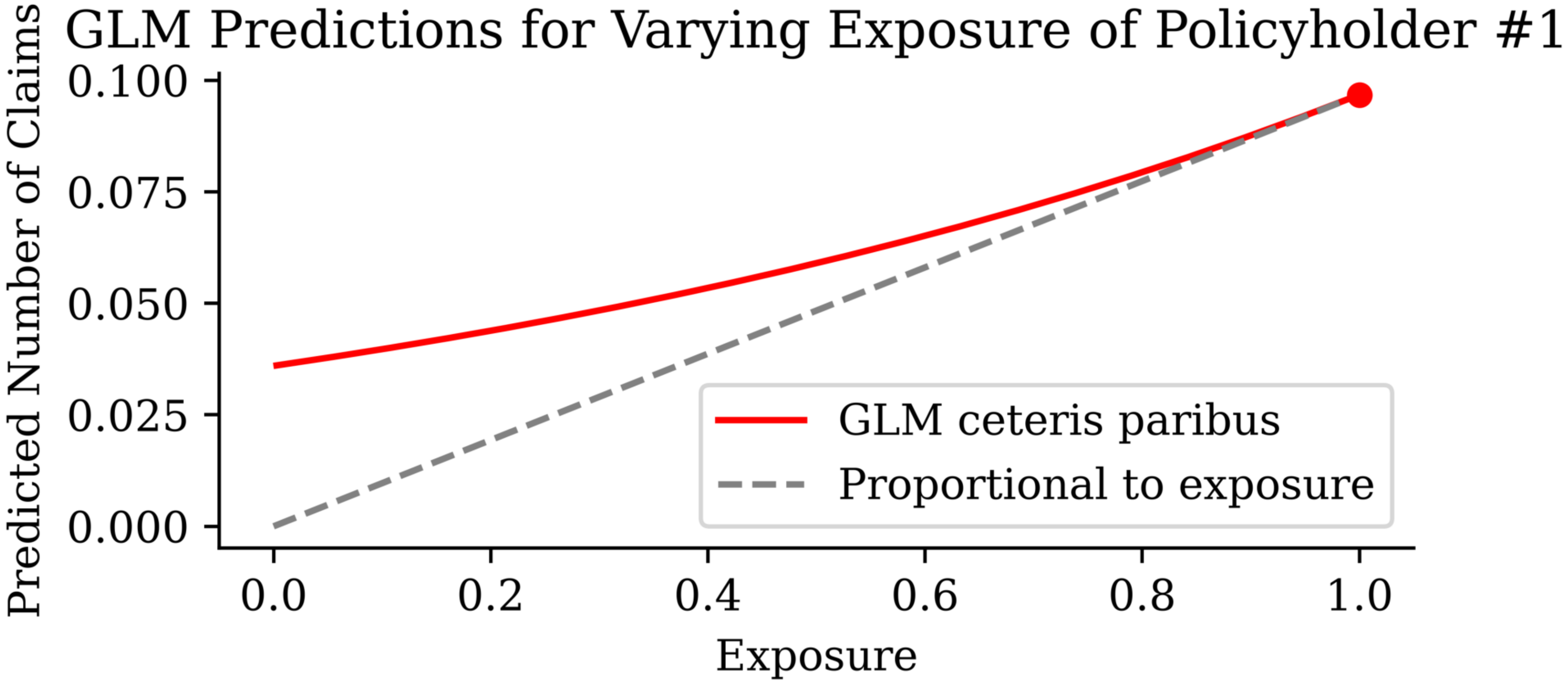
Zoomed out

We can see the trend if we zoom out to ages between 0 & 1,000.

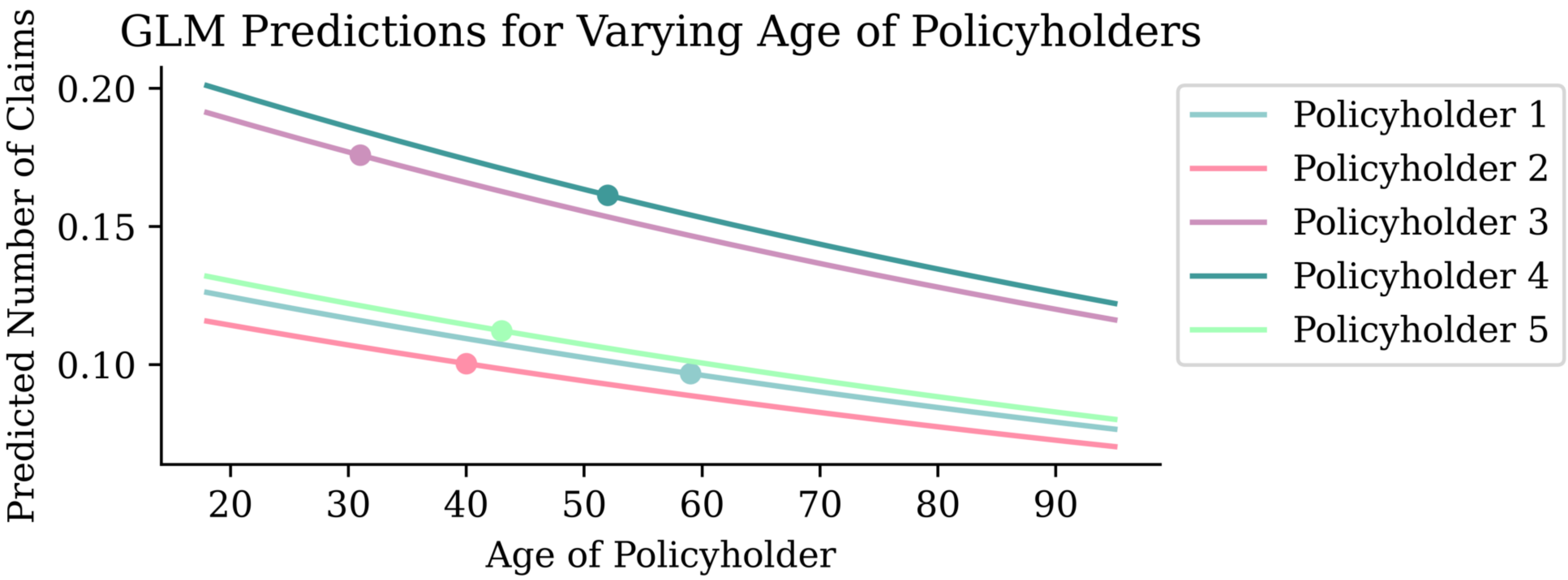


Interrogating an intentional bug

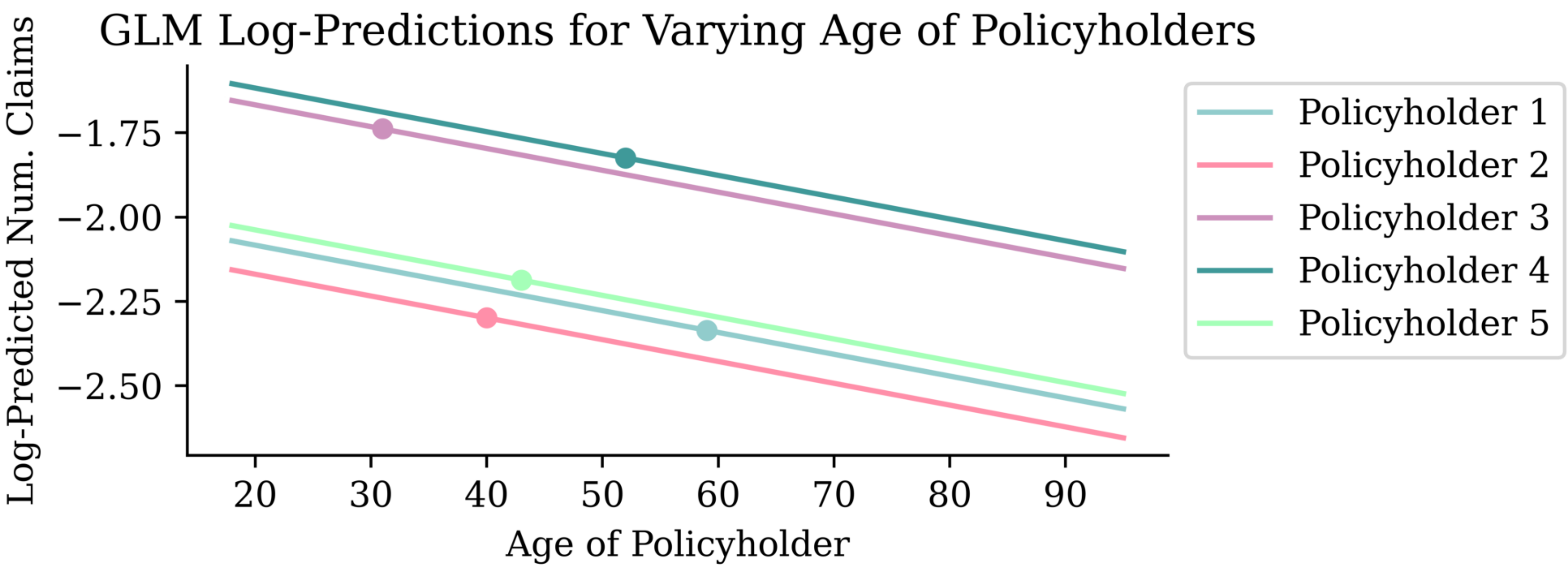
We fed `expo` in as an ordinary covariate — the CP plot exposes the consequence.



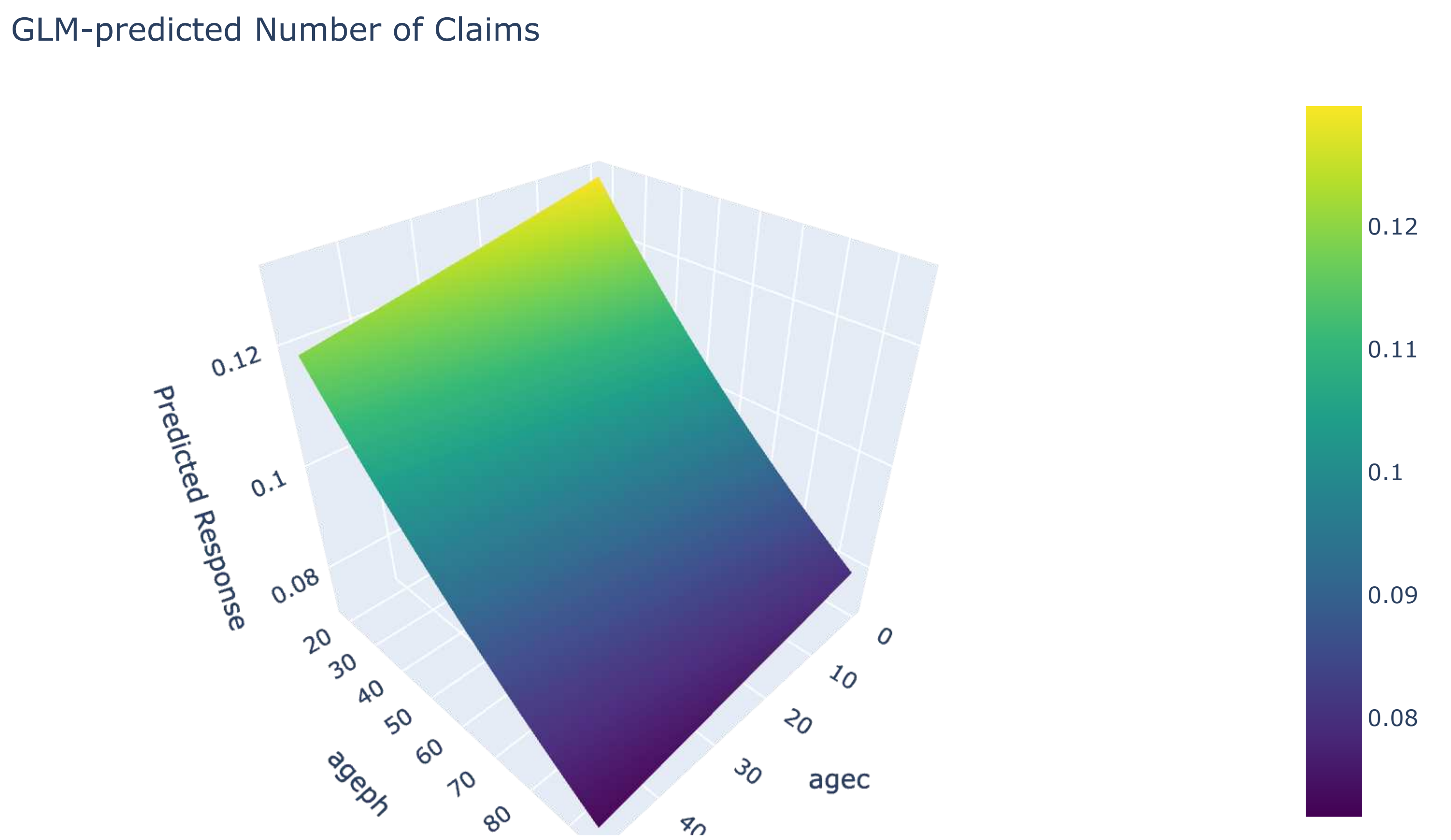
What if we look at multiple policyholders?



Logarithmic scale



Do it for ageph and agec



Generalised Additive Models (GAMs)

Transform a GLM's inputs nonlinearly, i.e.,

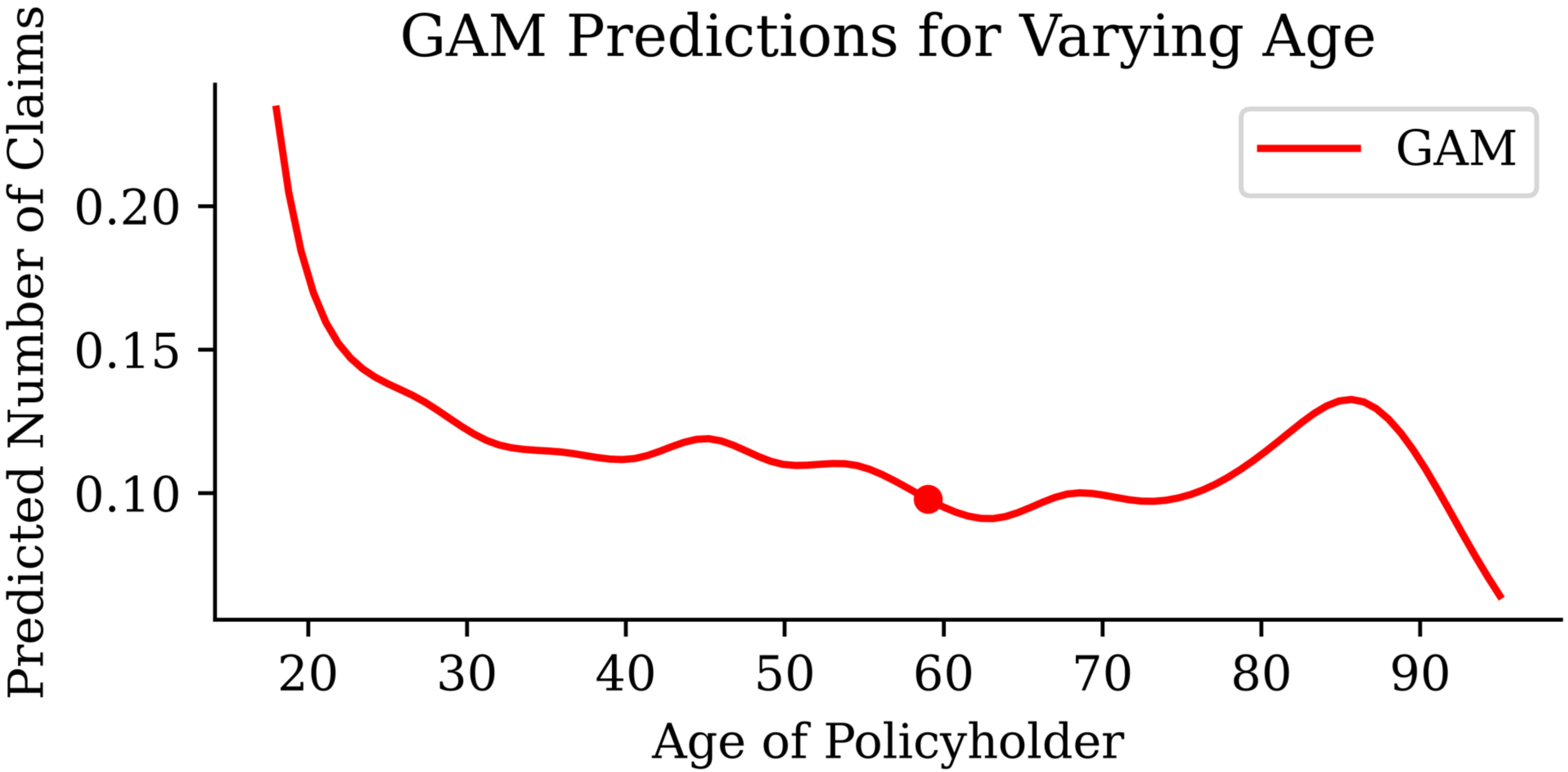
$$\hat{y}_i = g^{-1}\big(\beta_0 + f_1(x_{i,1}) + f_2(x_{i,2}) + \ldots + f_p(x_{i,p})\big)$$

The f_j are typically polynomials, step functions, or splines.

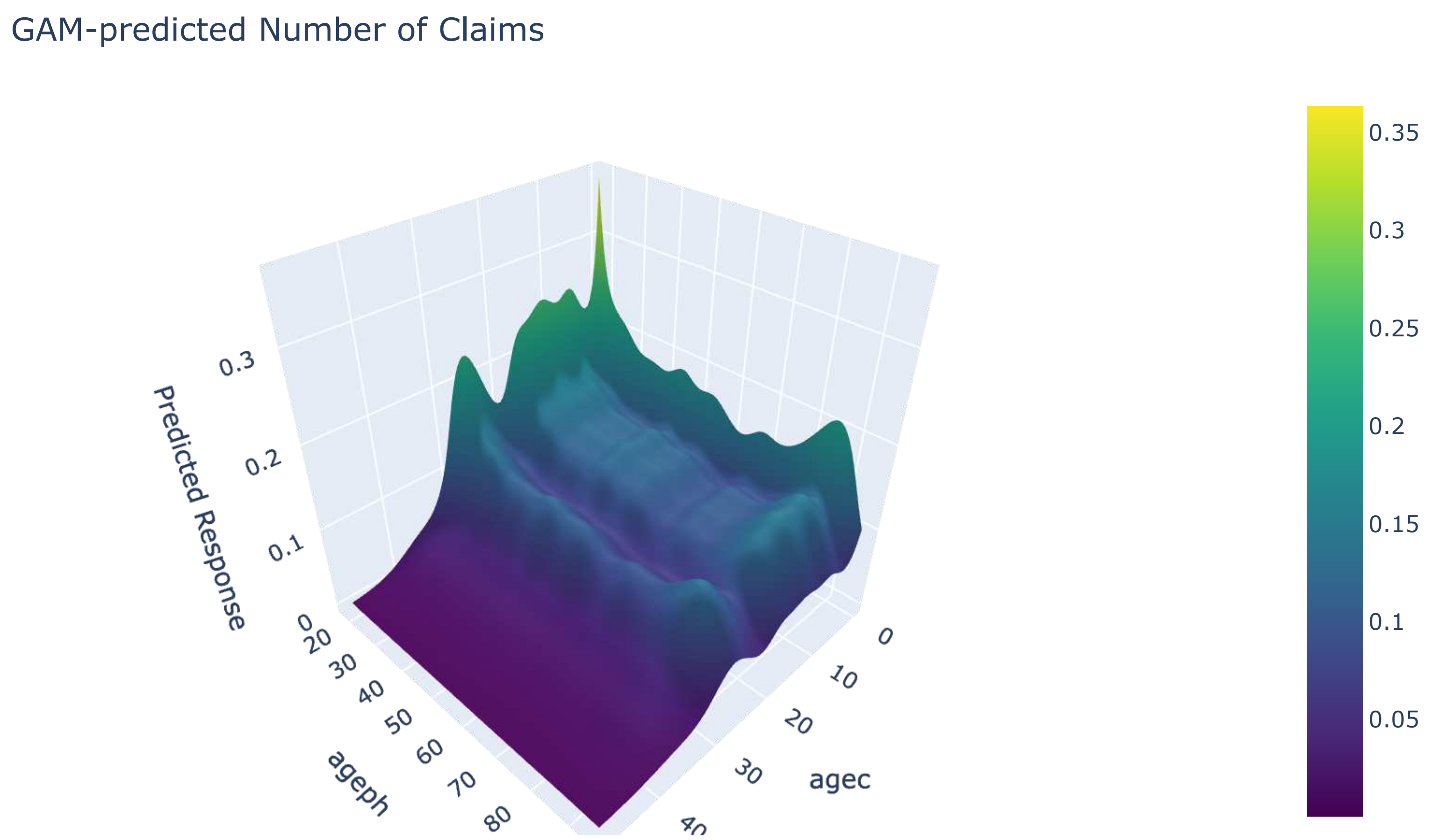
```
ct = make_column_transformer(
    ("passthrough", num_vars),
    (OrdinalEncoder(), cat_vars),
    verbose_feature_names_out = False
)
X_train_gam = ct.fit_transform(X_train_raw)
X_test_gam = ct.transform(X_test_raw)
y_train_gam = y_train_raw.values
y_test_gam = y_test_raw.values
```

```
formula = s(0) + s(1) + s(2) + s(3) + s(4) + s(5) + s(6) + \
    f(7) + f(8) + f(9) + f(10) + f(11)
gam_model = PoissonGAM(formula).fit(X_train_gam, y_train_gam)
```

Ceteris paribus plot for `ageph`



GAM bivariate effects for ageph and agec



Lecture Outline

Actuarial Neural Additive Model

ANAM, at its core, is a GAM where each f_j is a small neural network.

$$\hat{y}_i = g^{-1}\big(\beta_0 + f_1(x_{i,1}) + f_2(x_{i,2}) + \cdots + f_p(x_{i,p})\big)$$

The `anam` package fits these with a Poisson/Gamma likelihood, an exposure offset, optional *monotonicity* constraints, and automatic selection of features and pairwise interactions.

Source: Laub et al. (2026)

Specify the model

ANAM works on the raw data (standardising/encoding happens inside the model). A `smoothness` penalty is to help keep the continuous shape functions from overfitting.

```python
from anam import ANAM

anam_train, anam_val = train_test_split(train_raw, test_size=0.25, random_state=2000)

model_anam = ANAM(
    distribution="poisson",          # implies log link + Poisson NLL
    exposure="expo",                 # offset: log(expo) added to the linear predictor
    monotone={"bm": "increasing"},   # bonus-malus effect must be increasing
    smoothness=1e-3,                 # roughness penalty on continuous shape functions
)
```

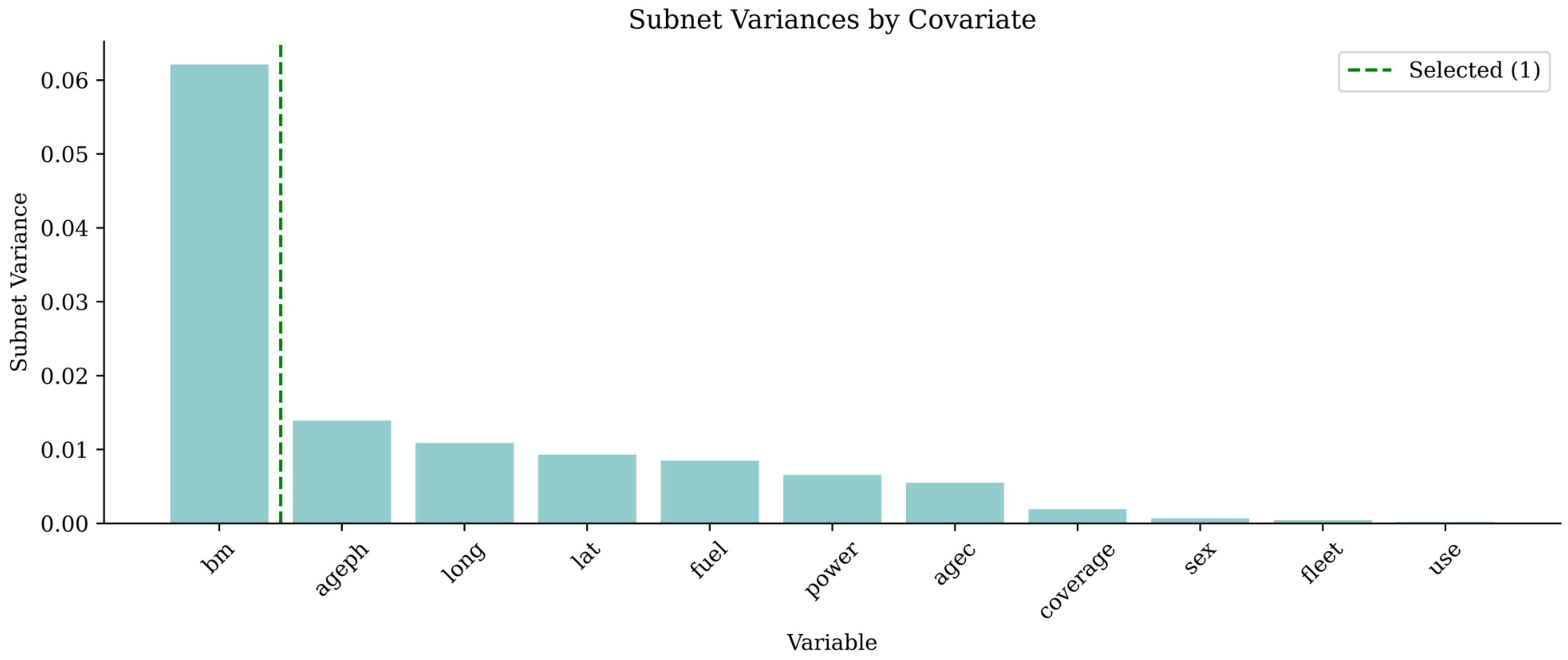
The idea is to choose the features and interactions semi-automatically.

Pick the main effects

Trains an ensemble of shallow NAMs and scores each covariate by the *variance* of its shape function.

```python
model_anam.rank_main_effects(anam_train, anam_train["nclaims"],
    X_val=anam_val, y_val=anam_val["nclaims"], random_state=2000)
```

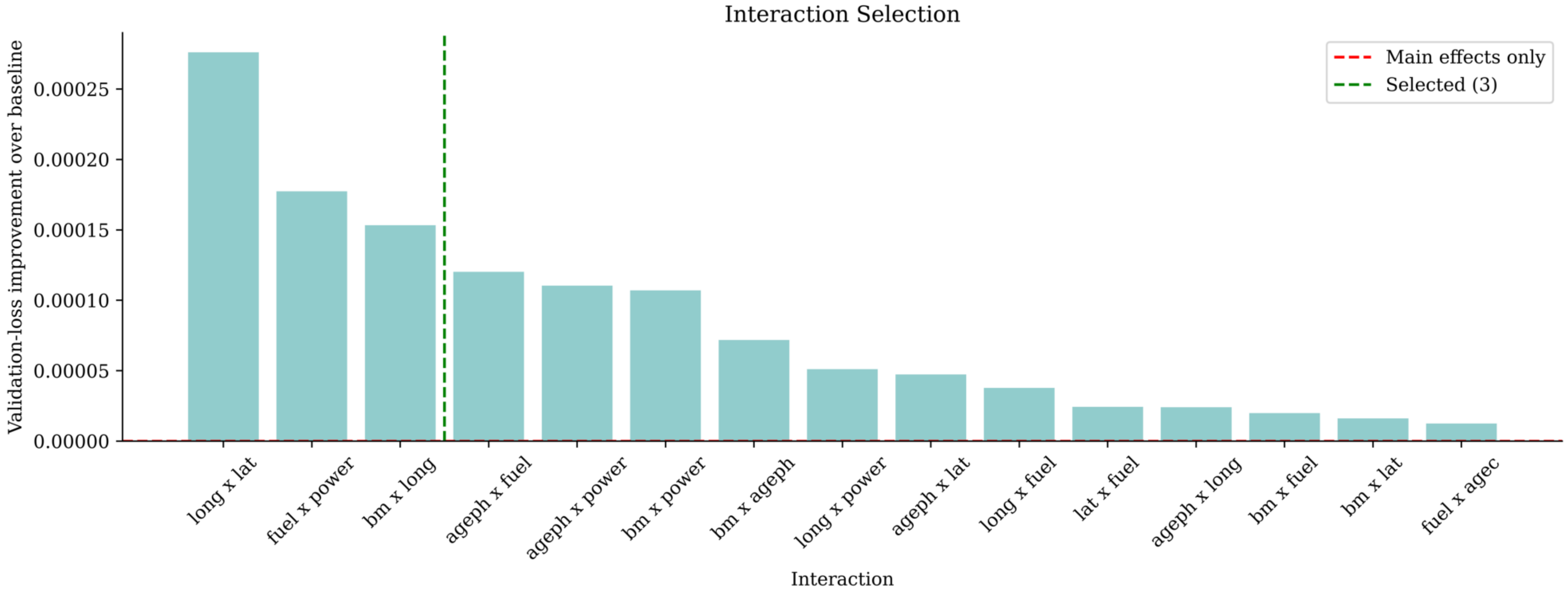
```python
model_anam.plot_main_effect_importance()
```



Pick the interactions

Scores every pairwise term that passes the *strong heredity* rule (a pair is only considered if both of its main effects are in the top n_main), measuring how much each candidate improves the validation loss over the mains-only baseline.

```
model_anam.rank_interactions(anam_train, anam_train["nclaims"],
    X_val=anam_val, y_val=anam_val["nclaims"],
    n_main=7, random_state=2000)
```



Lock in choices and fit

```python
model_anam.choose(n_main=7, n_interactions=6)
```

```python
model_anam.features_
```

```
['bm', 'ageph', 'long', 'lat', 'fuel', 'power', 'agec']
```

```python
model_anam.interactions_
```

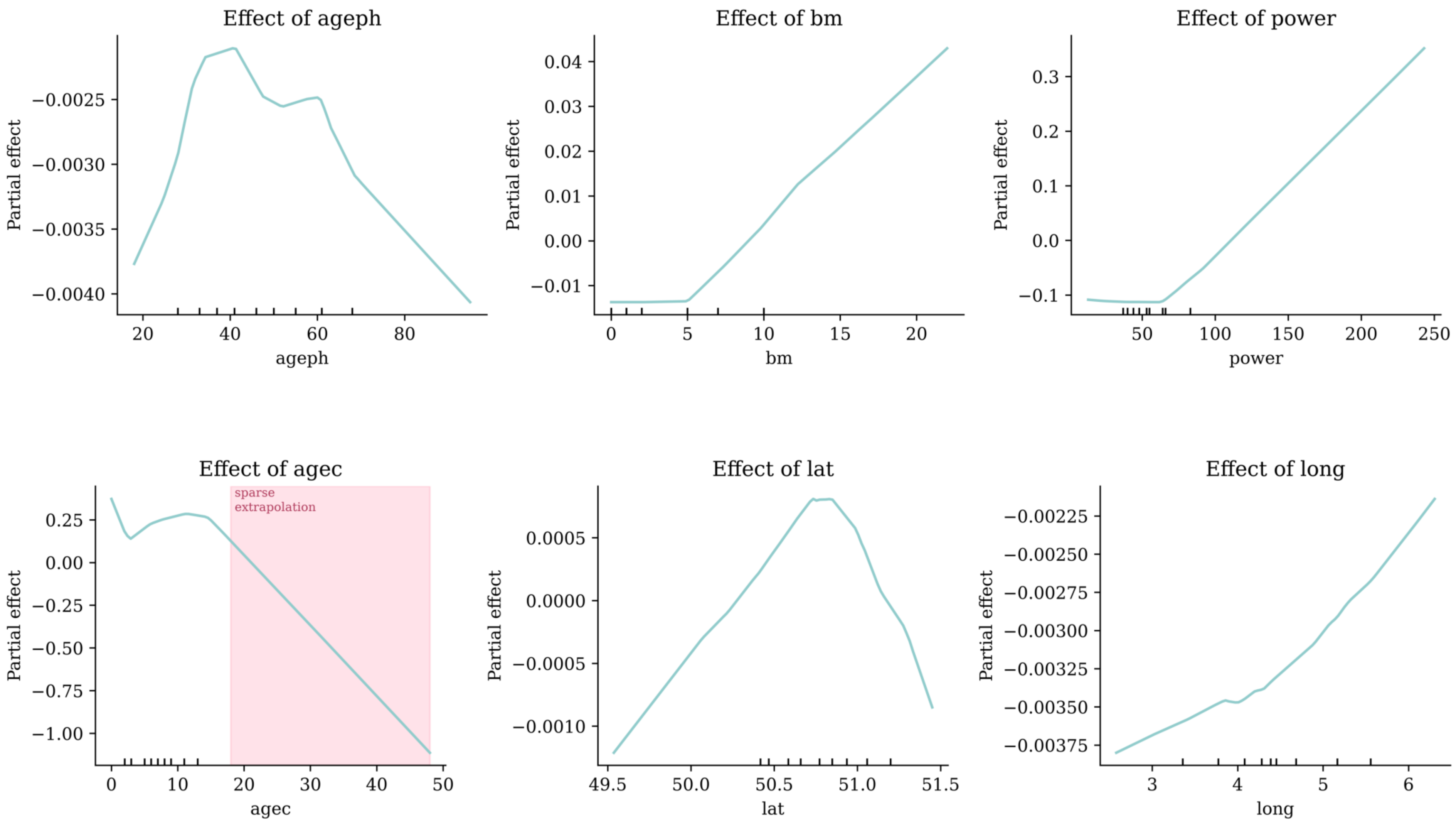
```
[('long', 'lat'),
 ('fuel', 'power'),
 ('bm', 'long'),
 ('ageph', 'fuel'),
 ('ageph', 'power'),
 ('bm', 'power')]
```

Exposure enters the final fit as an *offset*, not a feature: $\log \mu = \beta_0 + \sum_j f_j(x_j) + \log(\text{expo})$.

```python
model_anam.fit(anam_train, anam_train["nclaims"],
    X_val=anam_val, y_val=anam_val["nclaims"],
    epochs=300, batch_size=5000, lr=0.01, patience=40, random_state=2000)
```

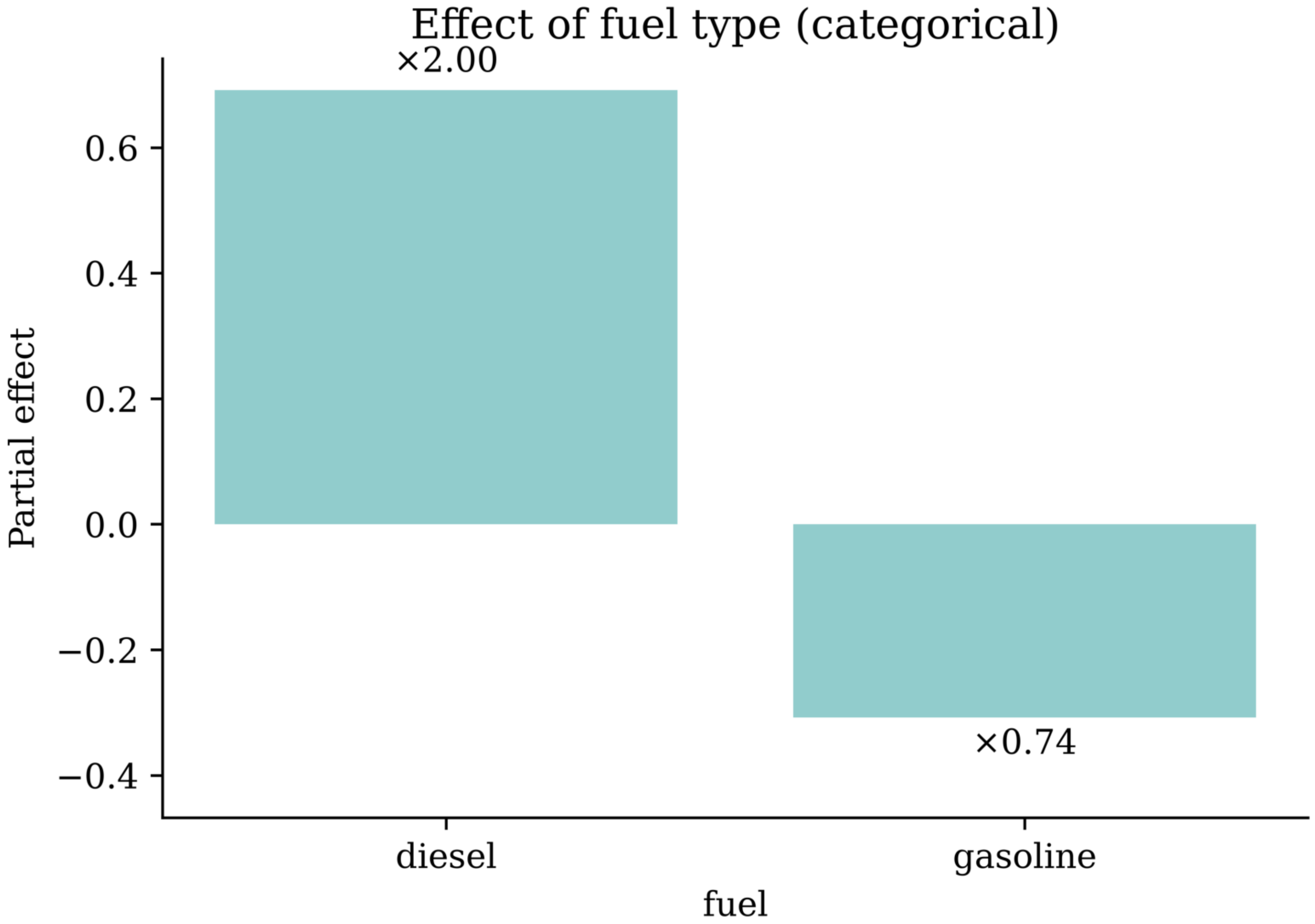
Reading the shape functions

Each curve is that feature's contribution to $\log \mu$; back on the original scale, $\exp(f_j)$ is a *multiplier* on the expected claim count.



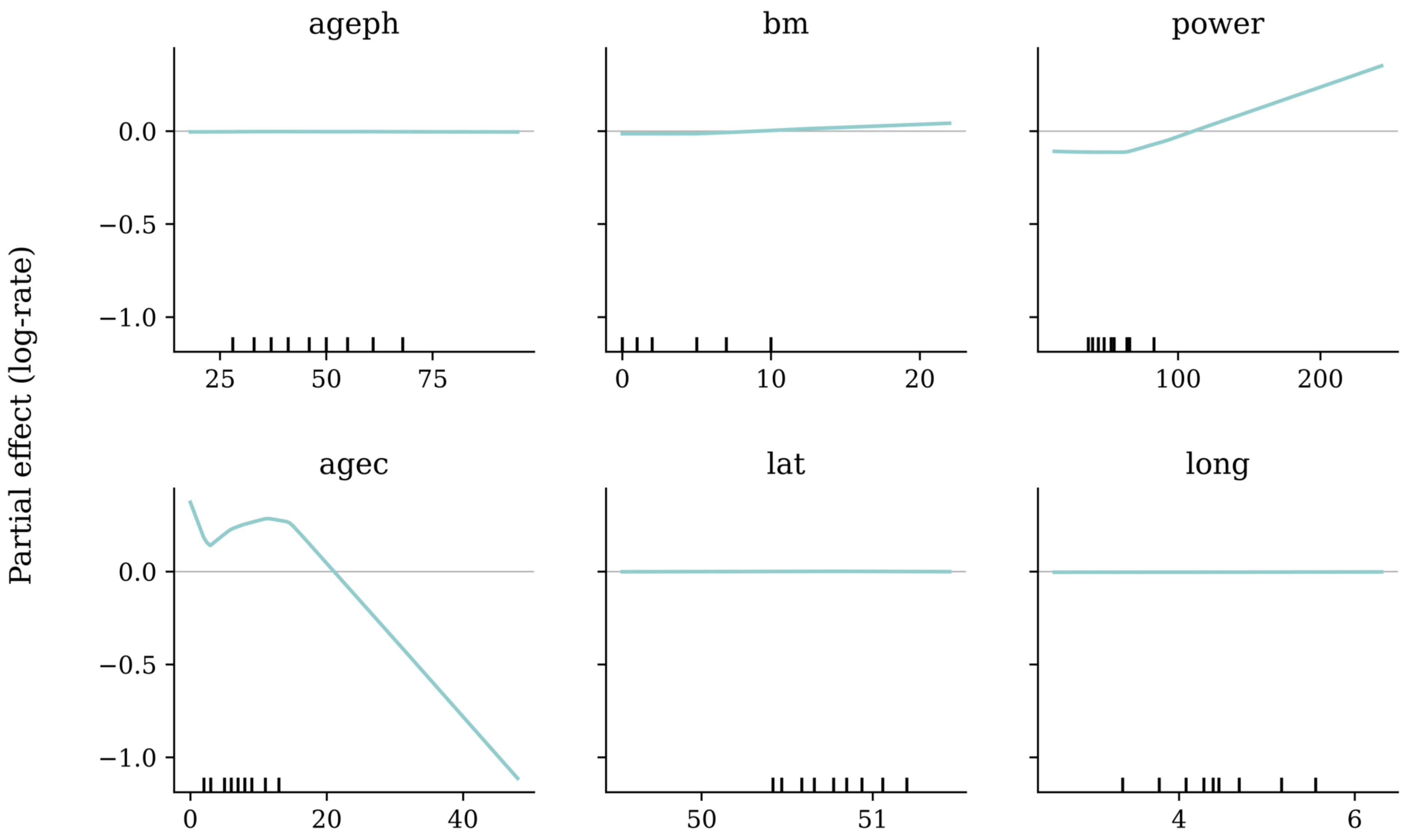
A categorical effect: fuel

Continuous features give *curves*; a categorical feature gives one value per level.



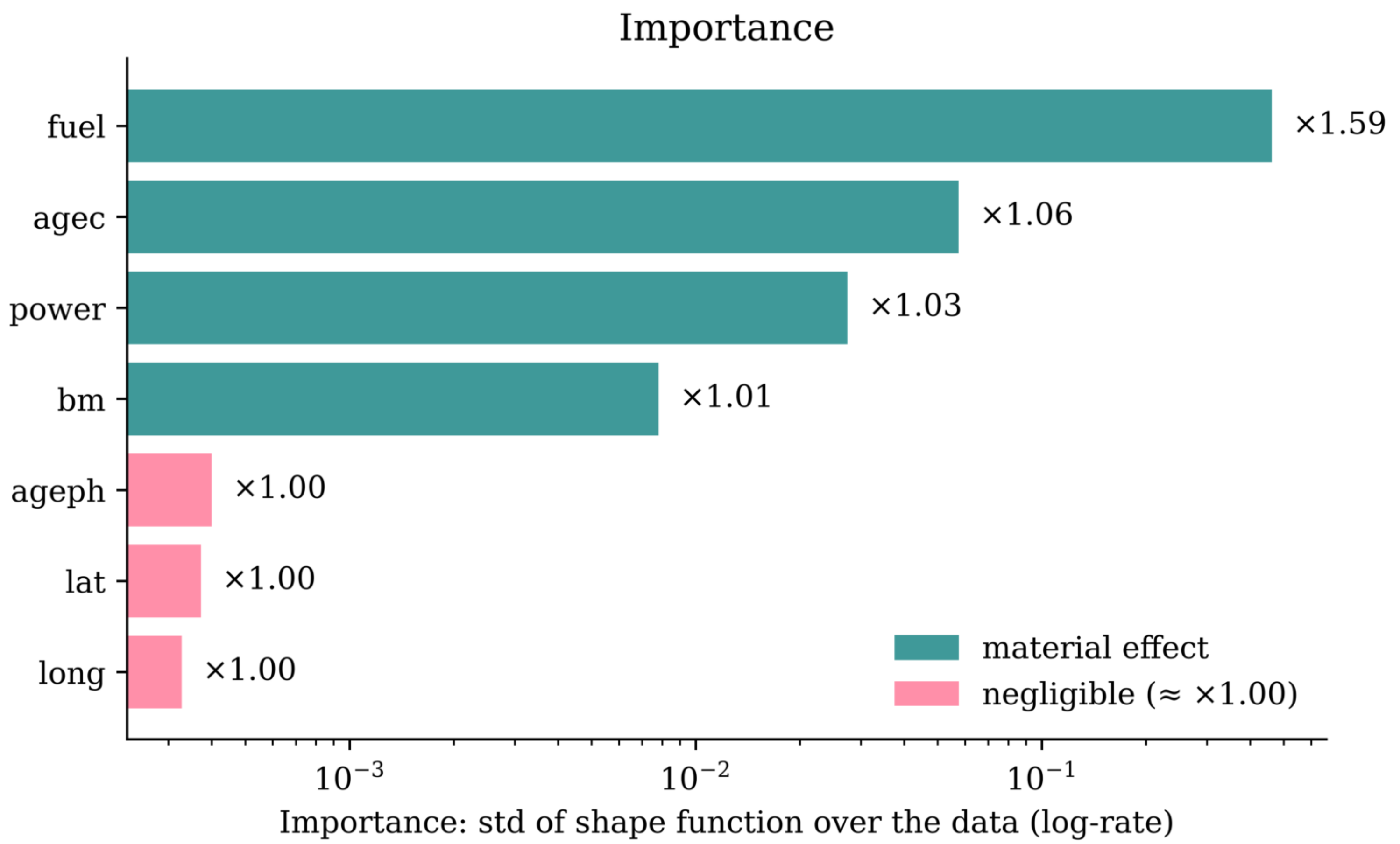
The same curves, one honest scale

Each panel above had its *own* y-axis. Put the six continuous effects on a *shared* scale and the picture changes: `agec` and `power` swing hard, while `ageph`, `lat` and `long` flatten to near-zero lines.

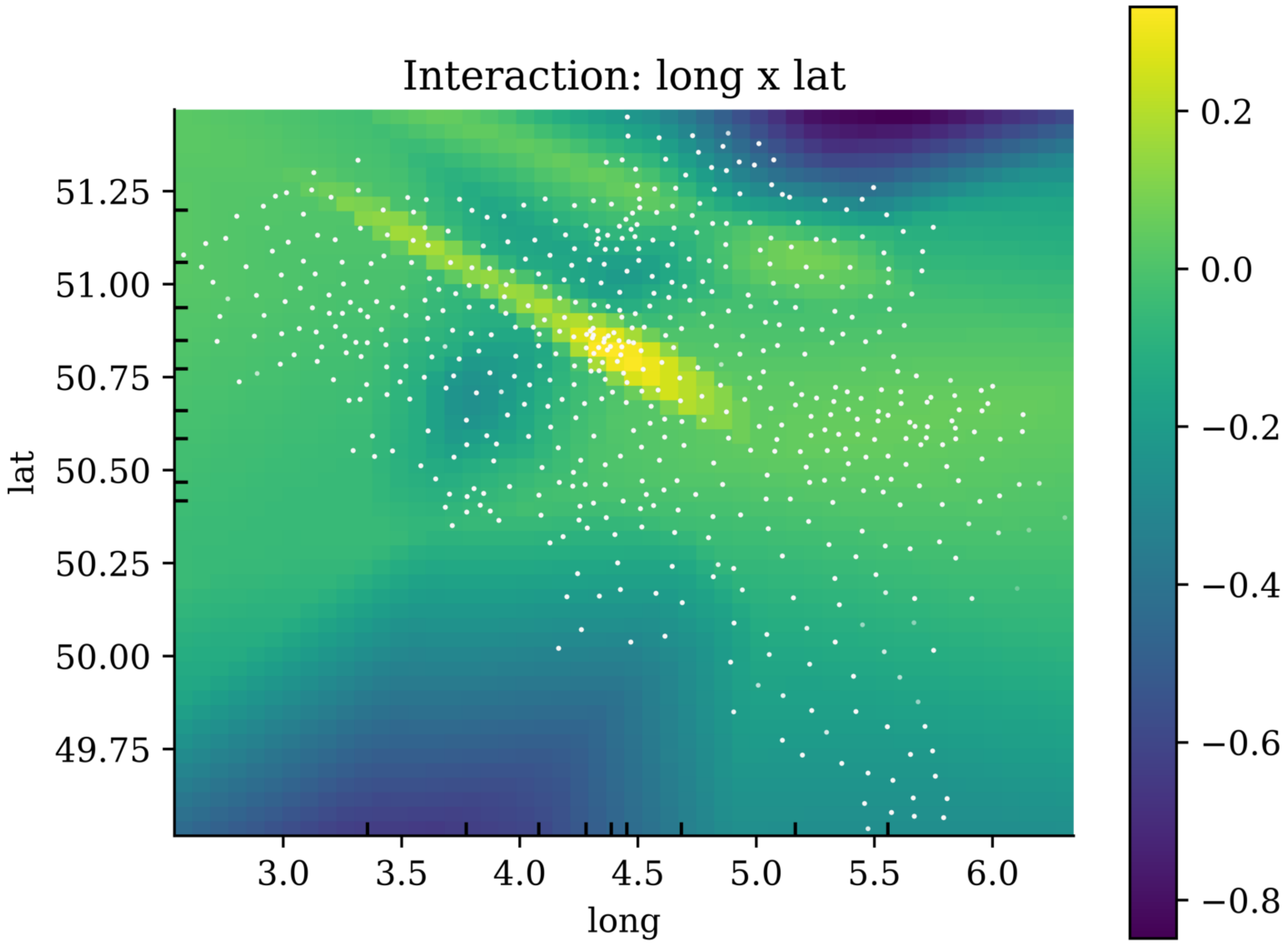


Which effects actually matter

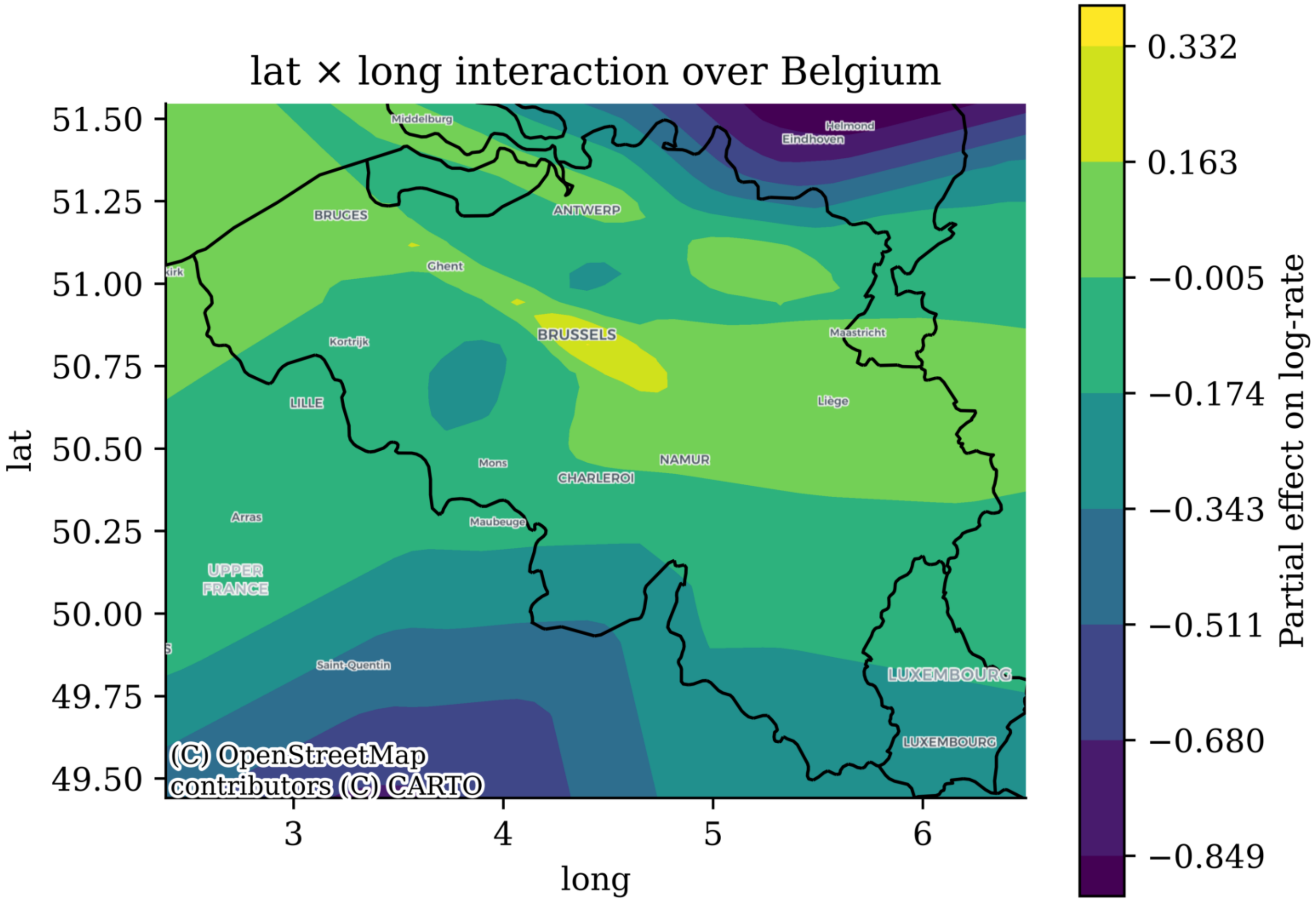
Measure the *standard deviation of each shape function across the training data*. Exponentiating gives a *typical multiplier* on the expected claim frequency.



An interaction effect



The interaction over a map

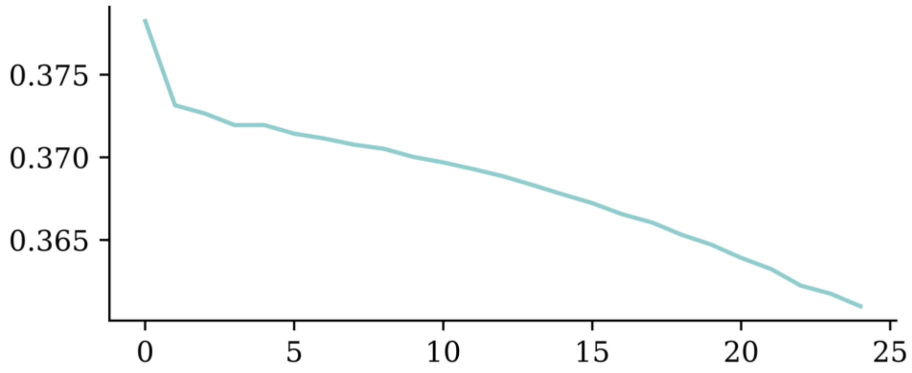


Lecture Outline

Build a neural network

```
ct = make_column_transformer(
    (StandardScaler(), num_vars),
    (OneHotEncoder(drop="first", sparse_c
    verbose_feature_names_out=False
)
ct.fit(X_train_raw)

X_train_nn = ct.transform(X_train_raw)
X_test_nn = ct.transform(X_test_raw)
y_train_nn = y_train_raw.values
y_test_nn = y_test_raw.values


random.seed(123)

nn_model = Sequential(
    [
        Input(shape=(X_train_nn.shape[1],
        Dense(128, activation="relu"),
        Dense(128, activation="relu"),
        Dense(128, activation="relu"),
        Dense(1, activation="exponential"
    ]
)
```

```
nn_model.compile(
    optimizer="adam",
    loss="poisson",
    metrics=["mae"]
)

history = nn_model.fit(X_train_nn, y_trai
    epochs=25, batch_size=64, verbose=0,
)
```



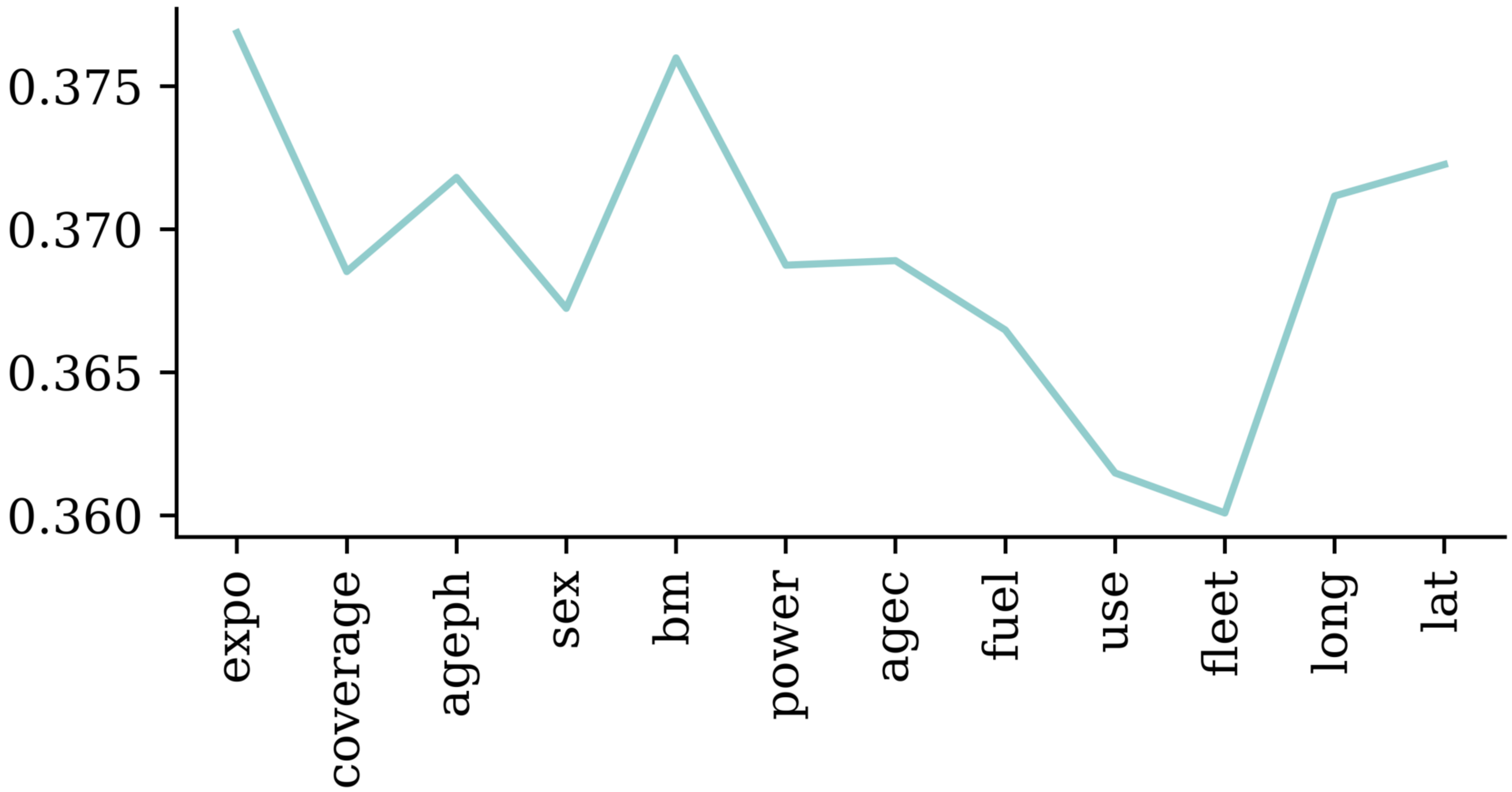
Permutation importance example

As recommended earlier, shuffle the *raw* columns and score the composition $f \circ t$, so each categorical is swapped wholesale and no “multi-hot” rows are created.

```python
def permutation_test_raw(model, ct, X_raw, y, num_reps=1, seed=42, batch_size=8192):
    """
    Run the permutation test on the raw features,
    scoring the composite model (preprocessing then network).
    """
    rnd.seed(seed)
    scores = []

    for col in X_raw.columns:
        original_column = X_raw[col].copy()
        col_scores = []

        for r in range(num_reps):
            X_raw[col] = rnd.permutation(X_raw[col].values)
            X_shuffled = ct.transform(X_raw).to_numpy()
            col_scores.append(
                model.evaluate(X_shuffled, y, verbose=0, batch_size=batch_size))

        scores.append(np.mean(col_scores, axis=0))
        X_raw[col] = original_column

    return np.array(scores)
```

Permutation importance example

```
scores = permutation_test_raw(nn_model, ct, X_train_raw, y_train_nn)
plt.plot(scores[:,0], label='Loss')
plt.xticks(ticks=np.arange(len(X_train_raw.columns)), labels=X_train_raw.columns, rotatio
```



PDP: Training data

Partial dependence plots start by looking at the training data.

```
X_train_raw
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	59.0	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	40.0	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	31.0	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	75.0	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	63.0	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	50.0	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```
nn_model.predict(ct.transform(X_train_raw), verbose=0).mean()
```

```
np.float32(0.12718096)
```

PDP: Alternate Reality

```python
X_train_pd = X_train_raw.copy()
X_train_pd['ageph'] = 18
X_train_pd
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	18	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	18	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	18	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	18	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	18	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	18	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```python
nn_model.predict(ct.transform(X_train_pd), verbose=0).mean()
```

```
np.float32(0.17916383)
```

PDP: Alternate Reality

```python
X_train_pd = X_train_raw.copy()
X_train_pd['ageph'] = 19
X_train_pd
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	19	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	19	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	19	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	19	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	19	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	19	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```python
nn_model.predict(ct.transform(X_train_pd), verbose=0).mean()
```

```
np.float32(0.17551786)
```

PDP: Alternate Reality

```
X_train_pd = X_train_raw.copy()
X_train_pd['ageph'] = 90
X_train_pd
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	90	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	90	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	90	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	90	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	90	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	90	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```
nn_model.predict(ct.transform(X_train_pd), verbose=0).mean()
```

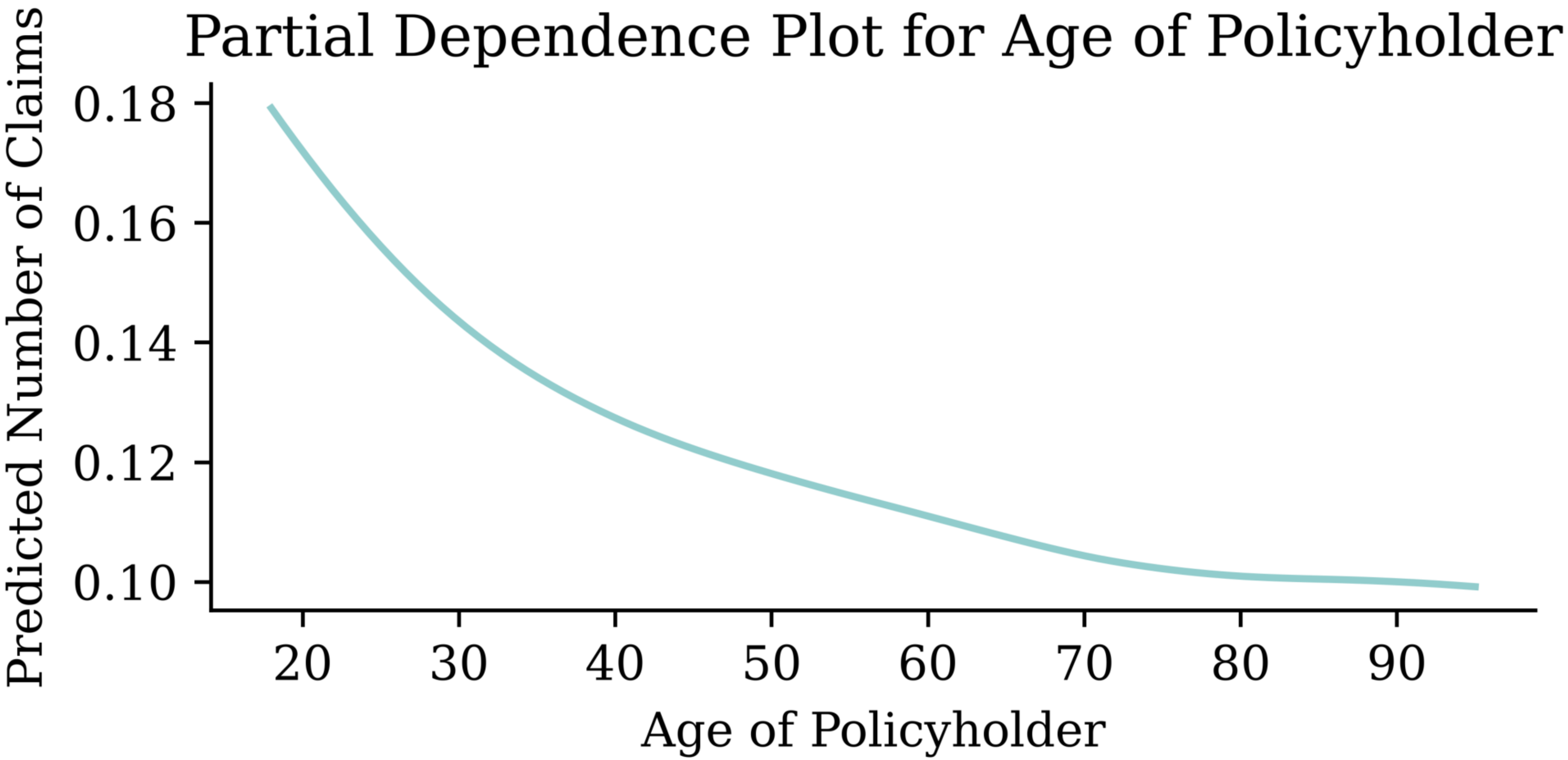
```
np.float32(0.100023024)
```

> **Question**
>
> What could go wrong?

Partial dependence plot code

```python
# Make partial dependence plot for ageph by hand
ageph_range = np.linspace(X_train_raw['ageph'].min(), X_train_raw['ageph'].max(), 100)
y_preds = []
X_train_pd = X_train_raw.copy()

for age in ageph_range:
    X_train_pd['ageph'] = age
    X_train_pd_ct = ct.transform(X_train_pd)
    y_pred_batch = nn_model.predict(X_train_pd_ct, verbose=0, batch_size=X_train_pd_ct.sh
    y_preds.append(y_pred_batch.mean())
```

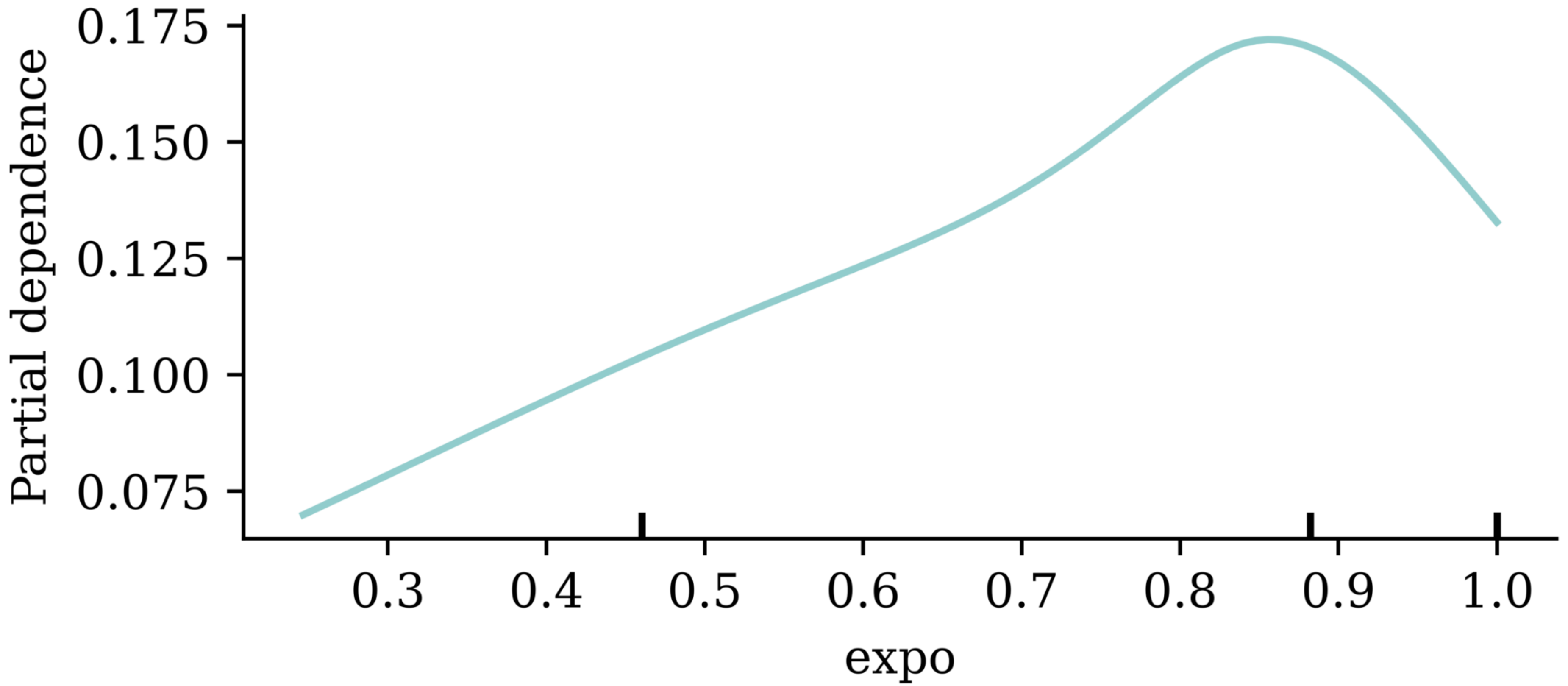
Partial dependence plot



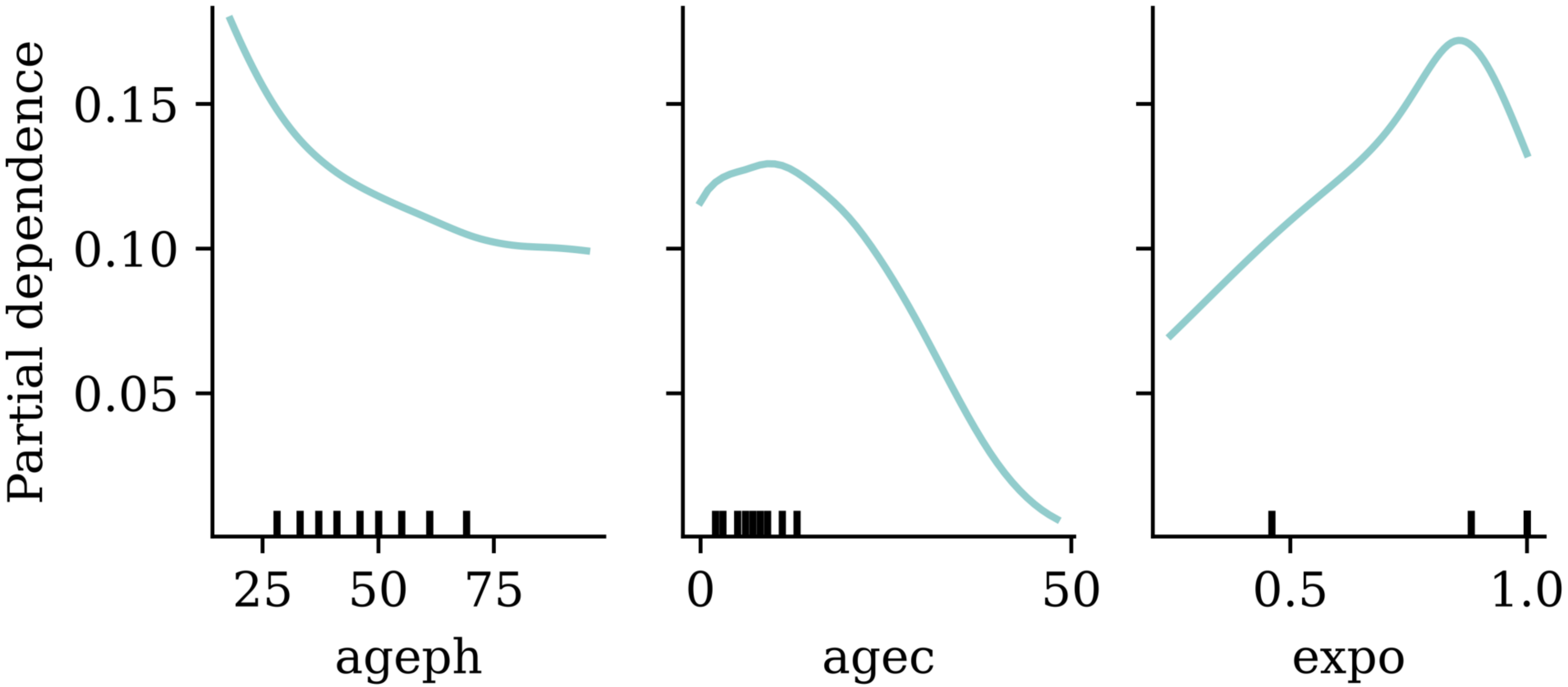
Using scikit-learn's PDP

(First need to trick scikit-learn a little..)

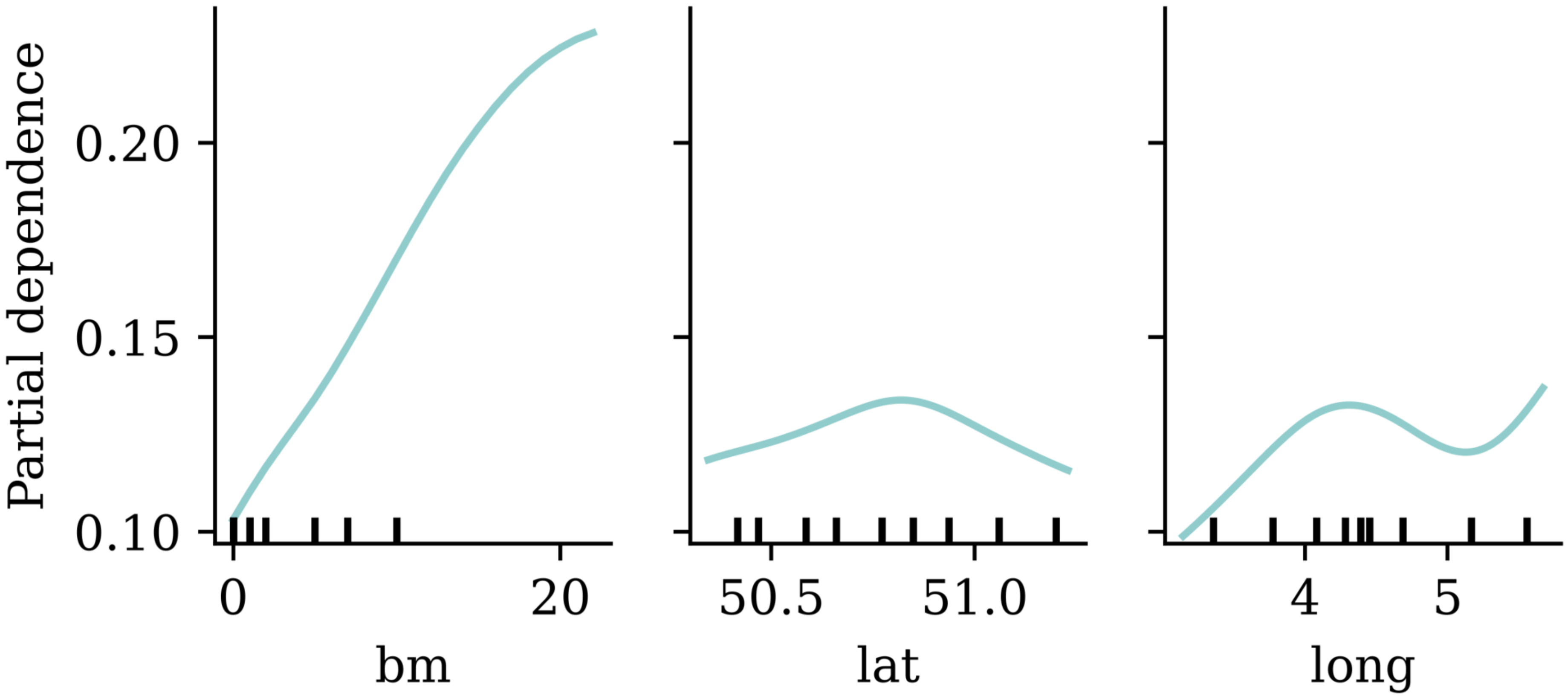
```
PartialDependenceDisplay.from_estimator(
    nn_model_skl,
    X_train_raw,
    features=[0],
    feature_names=X_train_raw.columns,
)
```



Explaining the ages and exposure

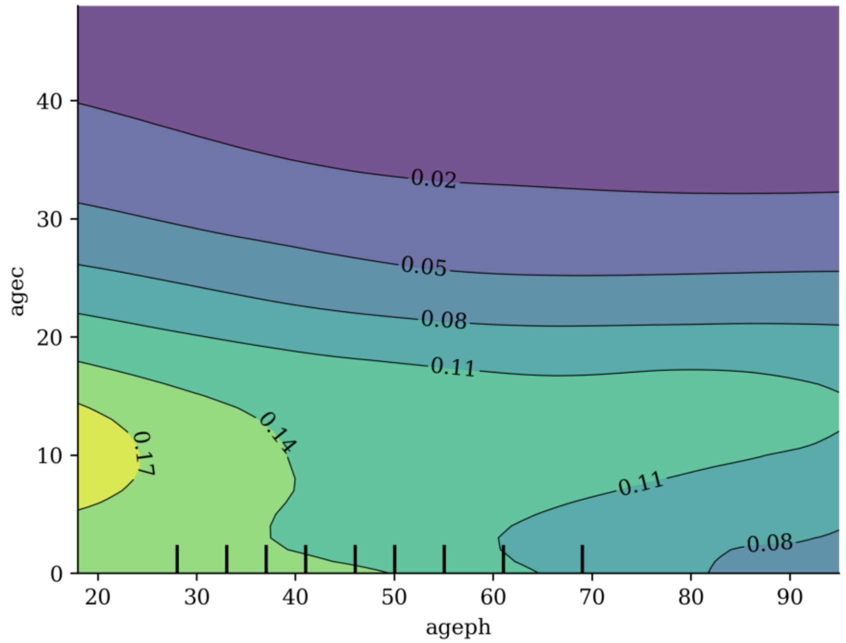


Explaining the bonus-malus and location



Bivariate age effects

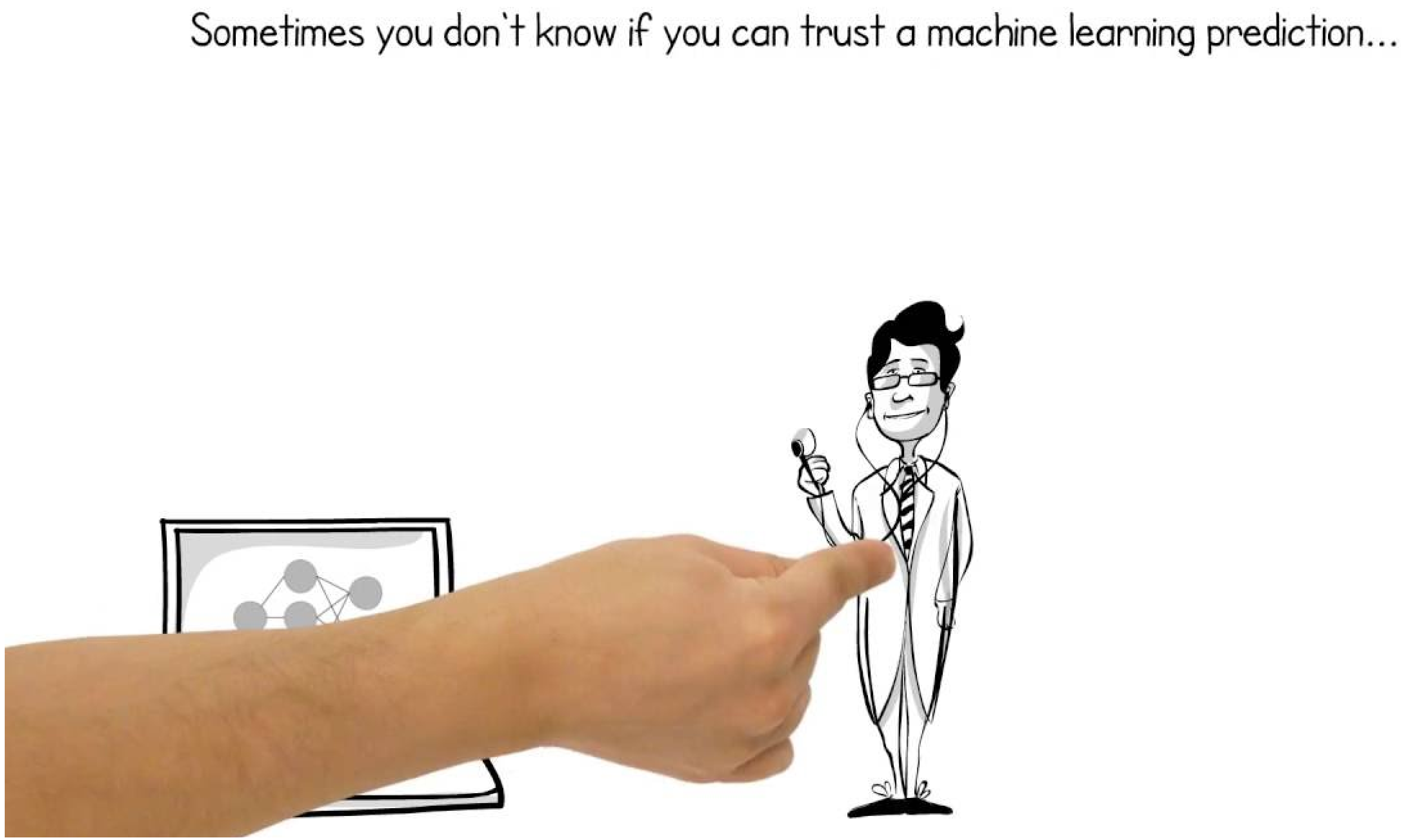
```python
cols = X_train_raw.columns.to_list()
inds = [cols.index(col) for col in ["ageph", "agec"]]

disp = PartialDependenceDisplay.from_estimator(
    nn_model_skl,
    X_train_raw,
    features=[tuple(inds)],
    feature_names=X_train_raw.columns,
)
```



Lecture Outline

LIME

Local Interpretable Model-agnostic Explanations employs an interpretable surrogate model (e.g. a linear regression) to explain locally how the black-box model makes predictions for individual instances.



SHAP Values

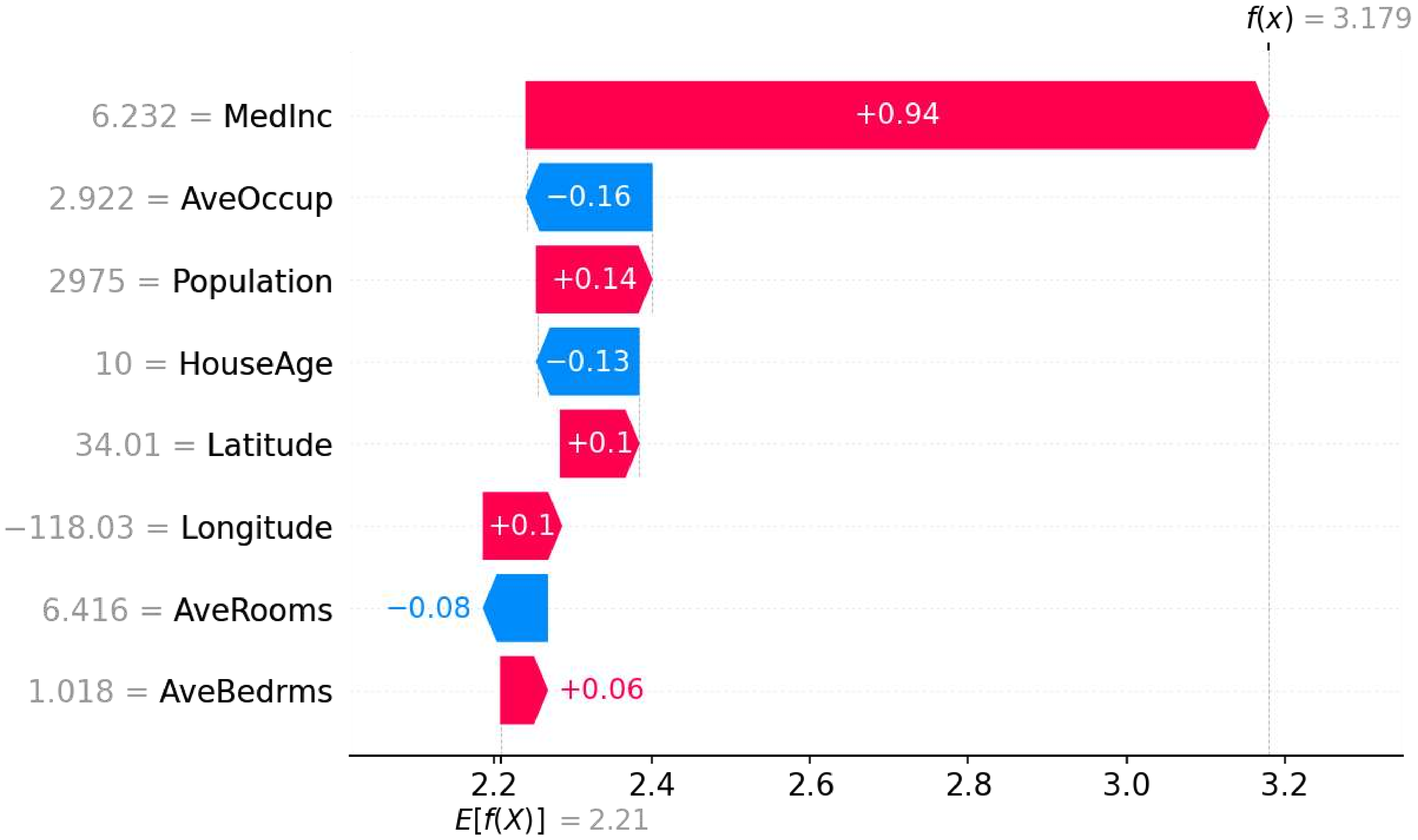
The SHapley Additive exPlanations (SHAP) value helps to quantify the contribution of each feature to the prediction for a specific instance (Lundberg & Lee, 2017).

The SHAP value for the jth feature is defined as

$$\text{SHAP}^{(j)}(\boldsymbol{x}) = \sum_{U \subset \{1,\dots,p\} \setminus \{j\}} \frac{1}{p} \binom{p-1}{|U|}^{-1} \big(\mathbb{E}[Y|\boldsymbol{x}^{(U \cup \{j\})}] - \mathbb{E}[Y|\boldsymbol{x}^{(U)}] \big),$$

where p is the number of features. A positive SHAP value indicates that the variable increases the prediction value.

SHAP waterfall plot



SHAP waterfall plot

Source: `shap` package documentation, *An introduction to explainable AI with Shapley values*

Counterfactual Explanations

> "A counterfactual explanation of a prediction describes the smallest change to the feature values that changes the prediction to a predefined output."
>
> — Molnar (2020)

Given a model f, an instance $\boldsymbol{x}$, and a desired prediction y', a counterfactual $\boldsymbol{x}'$ solves:

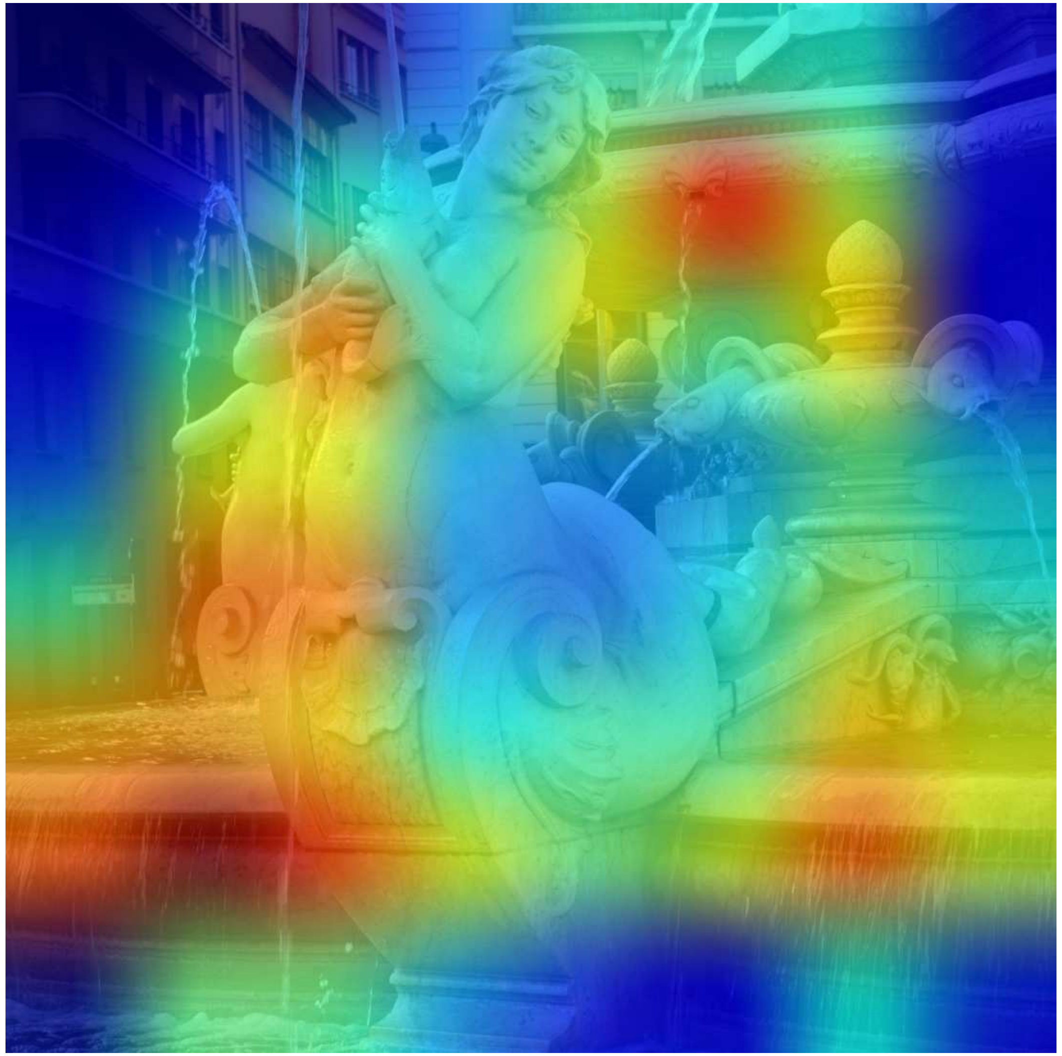
$$\boldsymbol{x}' = \arg\min_{\boldsymbol{x}'} \big(f(\boldsymbol{x}') - y'\big)^2 + d(\boldsymbol{x}, \boldsymbol{x}')$$

where $d(\cdot, \cdot)$ measures the distance between the original and counterfactual instances.

Note

For insurance pricing, a counterfactual might tell Bob: *"If your bonus-malus level were 5 instead of 12, your predicted premium would drop from $5,200 to $3,800."* This gives a contrastive explanation of why the prediction is higher. It is only actionable if the proposed change is feasible and under the policyholder's control.

Grad-CAM



Original image

Grad-CAM

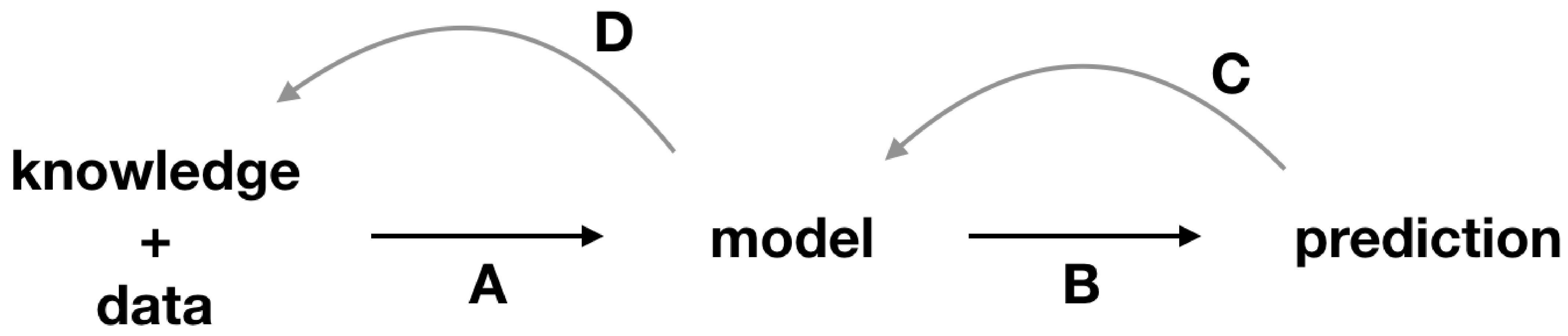
See Chollet (2021), Deep Learning with Python, Section 9.4.3.

See, e.g., Keras tutorial.

Lecture Outline

- Introduction
- Models With False Accuracy
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Other Local Methods
- **Conclusion**

Conclusion



"Figure 20.2: Explainability techniques allow strengthening the feedback extracted from a model. A, data and domain knowledge allow building the model. B, predictions are obtained from the model. C, by analyzing the predictions, we learn more about the model. D, better understanding of the model allows better understanding of the data and, sometimes, broadens domain knowledge." Biecek & Burzykowski (2021)

Package Versions

```python
from watermark import watermark
print(watermark(python=True, packages="contextily,geopandas,keras,matplotlib,numpy,pandas
```

```
Python implementation: CPython
Python version       : 3.14.5
IPython version      : 9.15.0

contextily : 1.7.0
geopandas  : 1.1.4
keras      : 3.15.0
matplotlib : 3.11.0
numpy      : 2.5.1
pandas     : 3.0.3
plotly     : 6.8.0
pygam      : 0.12.0
seaborn    : 0.13.2
scipy      : 1.18.0
shapely    : 2.1.2
sklearn    : 1.9.0
```

Glossary

- adversarial examples
- counterfactual explanations
- ceteris paribus plot
- global interpretability
- Grad-CAM
- individual conditional expectation (ICE) plot
- inherent interpretability
- LIME
- local interpretability
- partial dependence plot
- permutation importance
- post-hoc explainability
- SHAP values

References

Agarwal, R., Melnick, L., Frosst, N., Zhang, X., Lengerich, B., Caruana, R., & Hinton, G. E. (2021). Neural Additive Models: Interpretable Machine Learning with Neural Nets. *Advances in Neural Information Processing Systems*, *34*, 4699–4711.

Baeder, L., Brinkmann, P., & Xu, E. (2021). *Interpretable machine learning for insurance: An introduction with examples*. Society of Actuaries. https://www.soa.org/globalassets/assets/files/resources/research-report/2021/interpretable-machine-learning.pdf

Biecek, P., & Burzykowski, T. (2021). *Explanatory Model Analysis*. Chapman; Hall/CRC, New York. https://pbiecek.github.io/ema/

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32.

Carreira-Perpinán, M. A., & Tavallali, P. (2018). Alternating optimization of decision trees, with application to learning sparse oblique trees. *Advances in Neural Information Processing Systems*, *31*.

Charpentier, A. (2024). *Insurance, biases, discrimination and fairness*. Springer.

Chen, Z., Lu, Y., Zhang, J., & Zhu, W. (2024). Managing weather risk with a neural network-based index insurance. *Management Science*, *70*(7), 4306–4327.

Delcaillau, D., Ly, A., Papp, A., & Vermet, F. (2022). Model transparency and interpretability: Survey and application to the insurance industry. *European Actuarial Journal*, *12*(2), 443–484.

Doshi-Velez, F., & Kim, B. (2017). *Towards a rigorous science of interpretable machine learning*.

Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable’s importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, *20*(177), 1–81.

Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and harnessing adversarial examples. *arXiv Preprint arXiv:1412.6572*.

Laub, P. J., Pho, T., & Wong, B. (2026). An interpretable deep learning model for general insurance pricing. *Insurance: Mathematics and Economics*, 103270.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, *30*.

Molnar, C. (2020). *Interpretable machine learning*.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). Why Should I Trust You?": Explaining the Predictions of Any Classifier.